Automata Theory

Part of understanding computation involves the study of formal models of computation, which have traditionally been called automata.

CONTENTS

Motivation and History • Kinds of Automata • Classic Examples of Automata • Connections To Formal Languages • Connections To Computability Theory • Applications • Where to Lean More • Summary

Motivation and History

The field we now call automata theory arose from many different areas of research, including:

- **Mathematics**, through attempts to formalize the meaning of computation
- **Engineering**, through attempts to characterize (discrete) physical systems
- **Psychology**, through attempts to characterize human thinking and reasoning

These folks all came up with the notion of an **automaton** as a kind of machine, real or abstract, that follows a fixed set of instructions to carry out a step-by-step procedure.

A brief history of the field can be found at Wikipedia.

> ***Exercise**: Skim the Wikipedia article now.*

We want a rigorous theory of automata. Therefore, we will make the following assumptions about the automata that we study:

- A machine must have a **finite description**.
- The input to the machine is represented as **finite string of symbols** that come from **a fixed alphabet**.

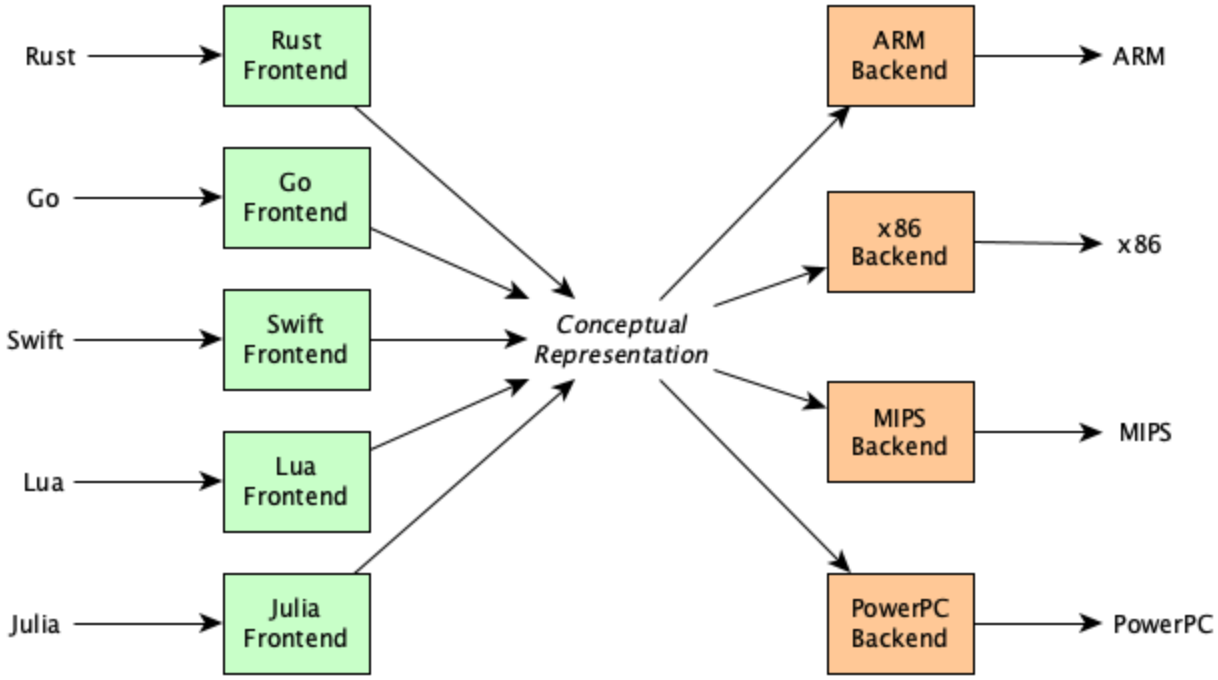
The first assumption is reasonable because a machine that requires an infinite description could not be built, let alone understood. The second is motivated by the fact that it's pretty difficult (impossible?) to conceive of any kind of information that we could not represent with strings of symbols. For example, the following capture quite a variety of information:

- `91772.3e-8`
- `Stay behind the yellow line!`
- `ᐊᐃᖏᓗᐃ ᓄᓇᕐᕕᐊᖅ`
- `/Pecs/Los Angeles/Berlin/Madrid//1 2/3 0/2 2/2 0/3 2///`
- `(* (+ 4 3) (-9 7))`
- `int average(int x, int y) {return (x + y) / 2;}`
- `<dog id="30024"><name>Lisichka</name><age>13</age></dog>`
- `{"type": "dog", "id": "30024", "name": "Lisichka", "age": 13}`
- `∀α β. α⊥β ⇔ (α•β = 0)`
- `1. f3 e5 2. g4 ♕h4++`

> ***Exercise**: Can you think of any information that cannot be represented as a string of symbols? If you can, how would you communicate it to someone else? What would "computing" with such information look like?*

Why create and study a theory of automata? Two reasons:

- It gives us a framework in which we can ask what computation is, what we can compute, what is necessary for computation, and how computations can be carried out.
- It provides insights into how we can design and build both real and abstract machines—and even programming languages! Abstract machines are quite important because they show how to carry out computations independent of any real machine and independent of any real high-level programming language, allowing us build multi-language compiler systems (where the abstract machine serves as a conceptual representation of a program):



Kinds of Automata

An automaton is any device, physical or abstract, that runs by itself (hence the prefix “auto”) to process some information. There should be no limits to your imagination regarding *how* an automaton should or should not look, nor what kinds of internal operations it carries out (as long as it is finitely describable).

People have created many kinds of automata. Some strive to be as minimal as possible (theoretically valuable, but hard to work with in practice); some are much more complex internally (but easier for humans to work with). In terms of technical details, there are at least four dimensions on which we can characterize automata:

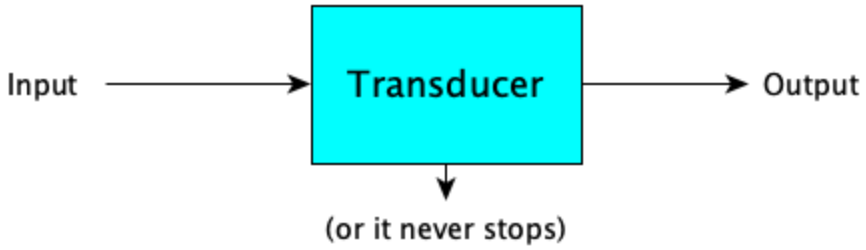
- (Behavior) Is the machine just answering a YES/NO question or computing a result?
- (Operation) How does the machine represent and execute each step of a computation?
- (Memory) Where are the inputs, outputs, and scratch data stored?
- (Architecture) Is the program hard-wired into the machine, or stored in memory with data?

There are more dimensions: synchronous vs. asynchronous, deterministic vs. non-deterministic, definite vs. probabilistic, classical vs. quantum, etc. But as these notes are introductory, we will focus only on the first four dimensions.

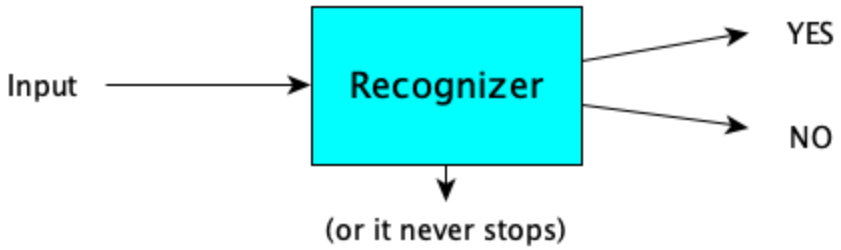
Behavioral Variations

Our first dimension is one of convenience only.

- A **transducer** automaton transforms an input into an output (if it finishes):



- A **recognizer** automaton accepts or rejects its input, by answering a YES-NO question about the input (if it finishes):



  If a recognizer machine M says YES to input w, we say that M **accepts** w.

  A recognizer that always finishes and answers YES or NO (and never loops) is called a **decider**.

Example: A example problem for a transducer might be: Compute the product of this pair of numbers.

Example: A problem for a recognizer might be: Is this number prime?

Operational Variations

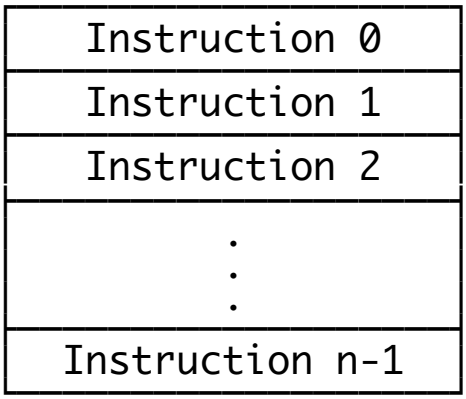
There are variations on how the automata are structured, particularly how their **control unit**, or algorithm, is specified. The main options are with a (1) a state machine, (2) a set of instructions, or (3) a set of rewrite rules.

- A **state machine** automaton performs a computation by moving through **states** that indicate in which phase of the computation one is in. The "program" is basically a **transition table**. The computation ends when you land in a state that doesn't have a transition out of the current state for the current machine configuration. Transition tables come in many forms, depending on the type of state machine, but are generally structured like so:

If you are in this state	and this event occurs	Then perform this action	and move to state

  If there can be more than one action/transition to take from a given state/input configuration, the machine is said to be **non-deterministic**, otherwise it is **deterministic**.

- An **instruction-set machine** has a control made up of a list of instructions. Normally instructions are executed in order, but some machine operands can be references to other instructions (e.g., via labels or by position) to allow conditional and iterative execution. Instructions that directly reference other instructions to explicity change the control flow are called **branches** or **jumps**.



- A **rewrite machine** is an automaton specified with a set of rewrite rules that transform the input to the output. For recognizers, we can have the output be a designated YES or NO symbol. The "program" defined by a rewrite system is something like:

If you see this substring	Rewrite with this substring

> ***Exercise**: Imagine non-deterministic vs. deterministic string rewriting machines. What issues come up in distinguishing these variations?*

Are these really all the same in some sense? Are these distinctions just for human convenience?

***Exercise**: Show how to represent a list-of-instructions machine as a state machine. Show how to represent a string rewriter as a state machine.*

Memory Variations

There's a lot of variety in memory, but two primary kinds are:

- **Tapes**: a tape is sequence of cells, each holding a single symbol. At any point the machine is looking at a **current cell** on the tape. The machine can move left or right only. The tape can have a finite number of cells, or can be infinite in one direction, or infinite in both directions. A machine can have multiple tapes. There are models where tapes are restricted to be read-only, write-only, stack-like (push/pop only), or queue-like (enqueue/dequeue only).



- **Registers**: A machine can have one or more registers. Each register holds some data. Usually in automata theory, a register holds a natural number (nonnegative integer) of unbounded size, but variations allow it to hold only a single symbol, or maybe a floating-point (but not real) number, or even a string. Registers are numbered $r_0, r_1, r_2, \ldots$ and they can be accessed by their number. Some models have a finite number of registers; in others, infinite. There can even be different classes of registers: for example, maybe only certain registers can do arithmetic, maybe only certain registers can be used for indirect addressing, etc.



What else? You can have fun, too, with things like multi-dimensional tapes. (A two-dimensional tape would allow for up, down, left, and right movements.) Use your imagination to build new computing machines.

Architectural Variations

One final variation of interest, applicable to instruction-based automata only, really, is: where are the instructions stored? There are two main architectures:

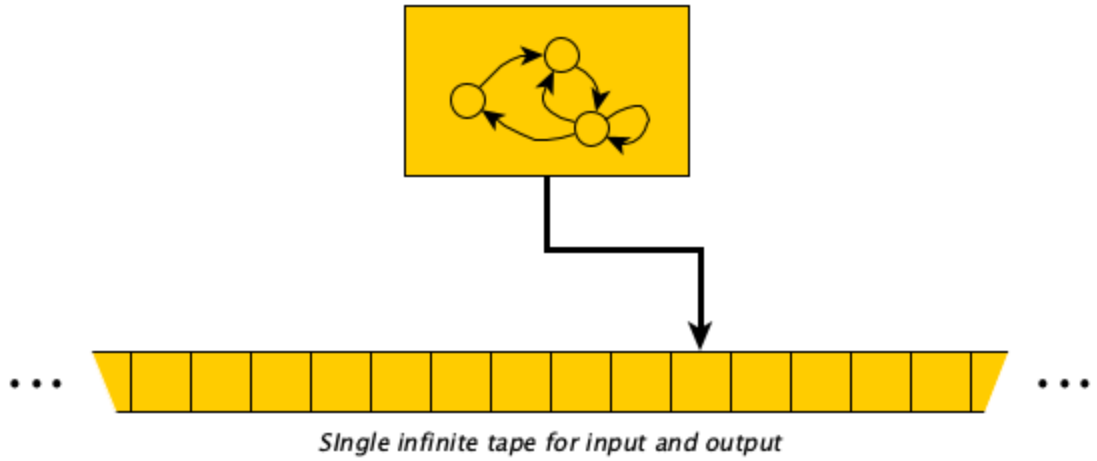
- **Harvard Architecture**: Instructions are stored in a dedicated, read-only instruction unit, separate from data.
- **Von Neumann Architecture**: Instructions are stored together with the data.

Classic Examples of Automata

Turing Machines (TMs)

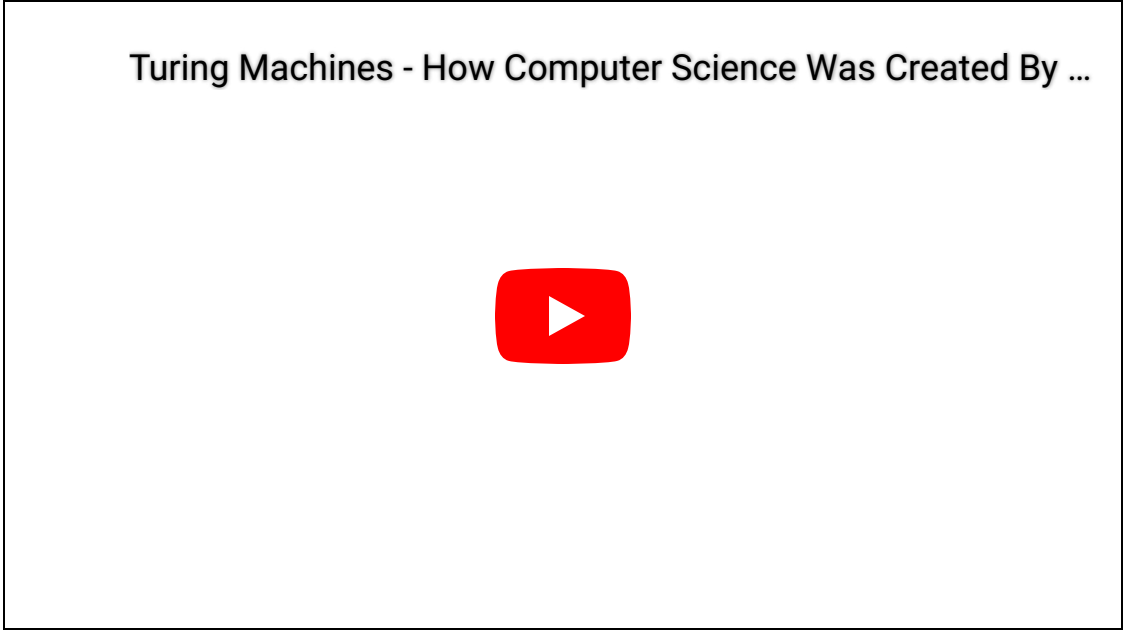
Turing Machines are so historically significant and influential, that I have an entire separate page of notes on them.

But the basic idea is that the input sits on a infinite tape of cells surrounded by blanks. The machine has a head pointing to the first symbol of the input. The machine's control is a finite set of rules that says "If in state q looking at symbol σ, replace the current symbol with σ', move either one cell right or left, and transition to state q'."

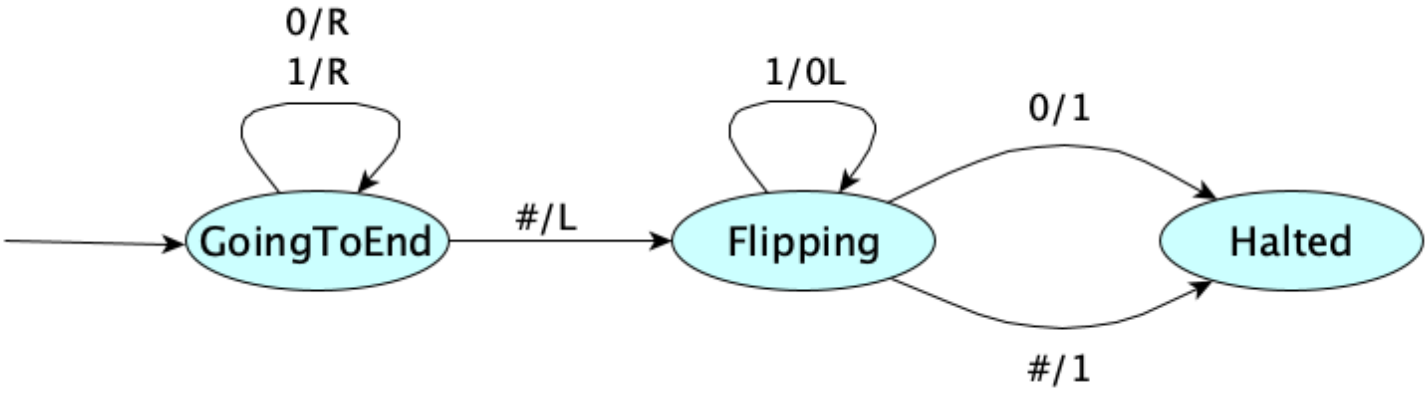


SIngle infinite tape for input and output

Watch Jade's video again:



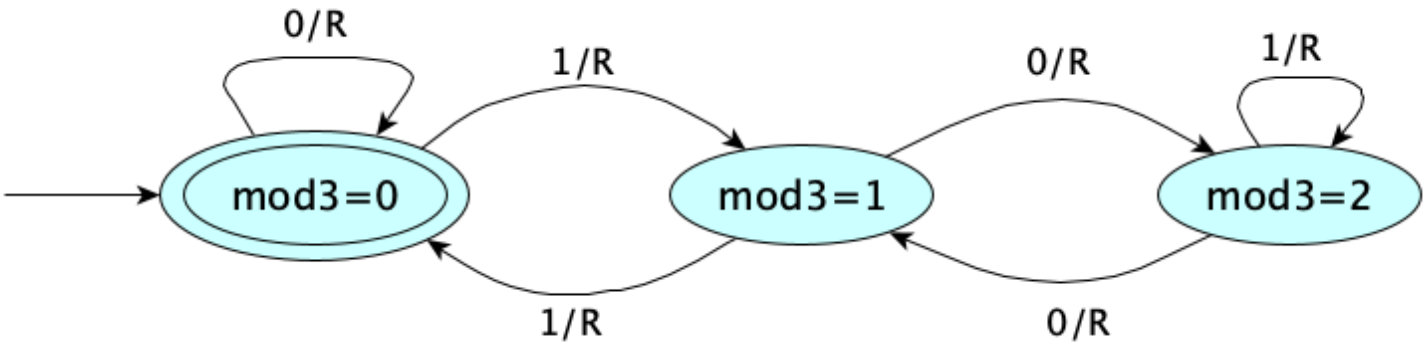
Again, we'll cover Turing Machines in detail later, but for now here's an example Turing Machine (of the transducer variety) to increment a binary number:



CLASSWORK

> Roleplay, with paper and pencil, the human computer of Turing's day sitting at a desk carrying out the task of incrementing the binary number 11010011111. Put yourself in each "mental state" expressed in the above machine.

And here is one (of the recognizer) variety to determine if a given unsigned binary number is divisible by 3:

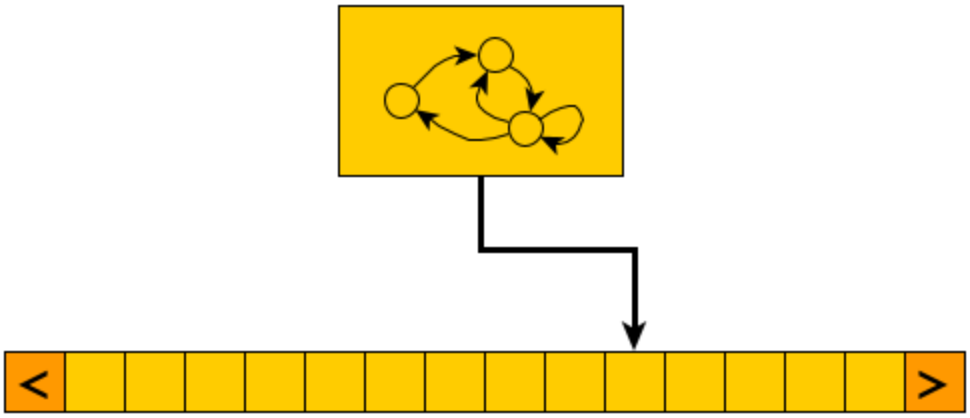


Later, we'll talk about how we came up with these beauties. Not too bad for us, right? But how the heck did Turing just figure all this stuff out to begin with? Pretty amazing, right? Yes, it *is* amazing even though his work was massively influenced by previous work of Gödel and Church and others, which Turing was happy to acknowledge.

> ***Wait, only one tape?***
>
> *Can a Turing Machine have multiple tapes? Sure. Turing was going for simplicity (if not outright minimalism) when we introduced this machine as a model for computation. You can show that any TM with multiple tapes can be simulated with a 1-tape TM, so various other tape-based models of computation can generally be considered kinds of Turing Machines. Some of these we'll see right now.*

Linear Bounded Automata (LBAs)

An LBA is a Turing Machine whose tape is not infinite but rather contains **only** its input with one cell to the left containing an non-replaceable "left-end-marker symbol" and one cell to the right containing a non-replaceable "right-end-marker" symbol.

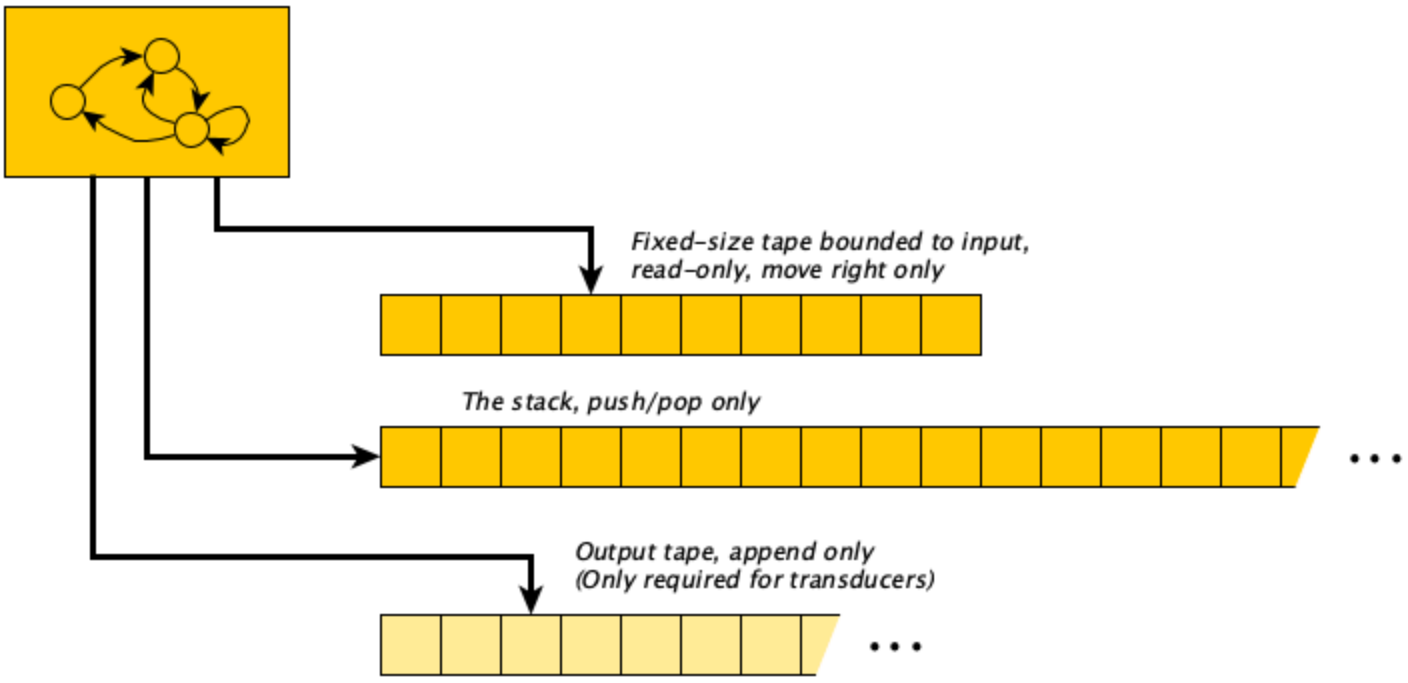


Fixed-size tape bounded to input string plus two cells for endmarkers

It feels a little more realistic, closer to a real computer, because of it having a finite "memory," right?

Pushdown Automata (PDAs)

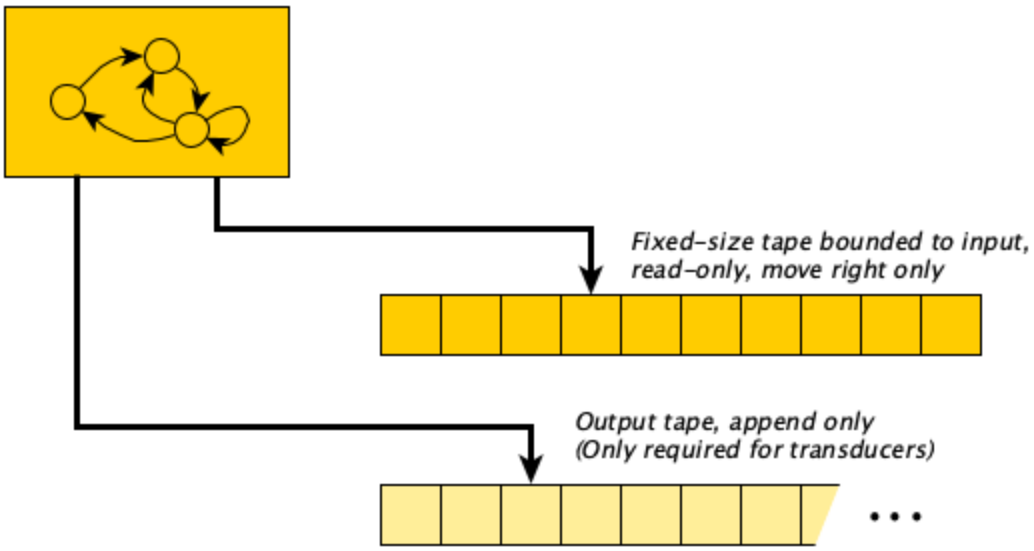
A PDA is an automata with a finite-state control, with a read-only input, and separate stack. At each step you read the next input symbol, and depending on the current top of the stack and the current state, you pop from the stack, push something new, and move to a new state.



***Exercise**: Why would anyone care about a machine like this? (Hint: how efficient are stacks, in terms of the Big-Θ complexity measure you learned about back in the day?)*

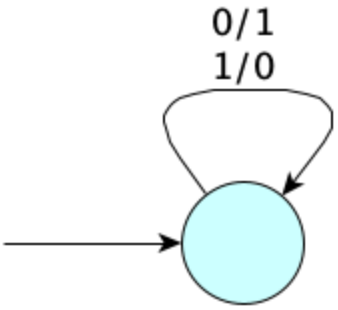
Type-3, or “Finite” Automata (FAs)

A **Type-3 Automaton**, often called just a **Finite Automaton** or **FA**, is a Turing Machine whose input is read-only and that only moves to the right on each step. For transducing FAs, the output is written to a separate output tape that is write-only, initially blank, and only appended to.

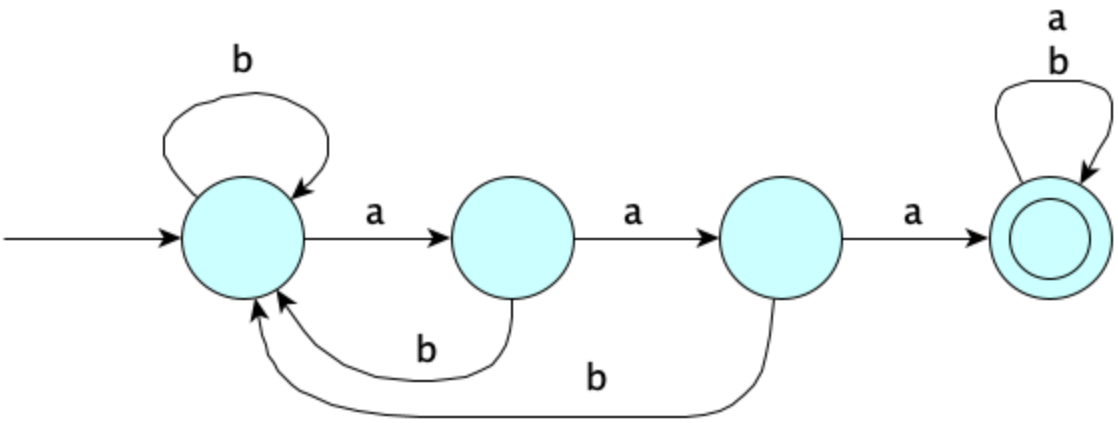


> ***Do not let this terrible name confuse you***
>
> *It is unfortunate that these machines are called finite automata, because Turing Machines have a finite number of states, as do LBAs and PDAs. Maybe it is because of finite memory? No LBAs have finite memory. No one has a good name for this class of automata. Everyone calls them FAs.*
>
> *Perhaps if you never heard of LBAs the name might make sense.*
>
> *By the way, you may sometimes hear them called FSMs for “finite state machines” but that is much, much worse since all reasonable automata have a finite number of states and the acronym is easily confused with the deity of Pastafarianism.*

When describing FAs, we don’t have to worry about blanks, or say whether we are moving left or right! All we need is the input symbol and the output symbol(s). For a first example, here is a transducer FA that computes the one’s complement of its input:



For acceptors, we don't have any output at all, so the machine transitions just have the input symbol. Here's an FA to determine whether its input has three consecutive ***a***'s somewhere:



An FA that writes to its output on a transition is called a **Mealy Machine**. You can also make an FA transducer that outputs a symbol whenever it enters a state; this would be called a **Moore Machine**.

> ***Exercise**: What "kind" of computations do Finite Automata represent?*

Queue Automata

A queue automaton is a TM that, instead of moving left or right at each step, instead always dequeues the symbol at the left and enqueues some other string of symbols on the right.

$\mathcal{P}''$

An interpreter for the $\mathcal{P}''$ language can be considered an automata. This is an exceedingly low-level and minimal notation. Check it out at Wikipedia.

Brainfuck

Like $\mathcal{P}''$, Brainfuck (or Brainf**k as it is commonly known) is a programming language with an explicit machine execution model. It is much more widely known and way easier to understand than $\mathcal{P}''$, as it is less minimal.

The execution model features (1) a fixed program stored on a read-only tape where each cell holds one of eight instructions, (2) a read-only input stream of bytes (you can think of it as a tape), (3) a write-only output stream of bytes (also a tape), and (4) a data tape with a fixed left end and infinite to the right (although really the language specification says at least 30,000 cells). The eight instructions are:

Instruction	Description
+	Increment the byte in the current data cell
−	Decrement the byte in the current data cell
.	Output the byte in the current data cell
,	Read next byte from input and store in current data cell
<	Move left on tape
>	Move right on tape
[	If byte in current data cell is 0, jump to instruction right after the next *matching*]
]	If byte in current data cell is not 0, jump to instruction right after the previous *matching* [

Here's a program in the language. Can you figure out what it does?

```
+++++++[>+++++[>++>+++<<-]<-]>>++.>.
```

▶ Need a hint?

See the Language overview at Esolang, the Wikipedia article, and this collection of example programs.

String Rewriting Systems

We can mechanically compute by applying rules (from a finite set of rules) that allow us to rewrite part of a string into another. We start with the input surrounded by begin and end markers `[` and `]`, and keep rewriting, non-deterministically, there are no more rules to apply.

For example, here's an SRS for incrementing binary numbers:

$0]$	$\to$	$Y1$
$0Y$	$\to$	$Y0$
$1Y$	$\to$	$Y1$
$[Y$	$\to$	ε
$1]$	$\to$	$X0$
$0X$	$\to$	$Y1$
$1X$	$\to$	$X0$
$[X$	$\to$	1

Some example runs:

- $[0] \Rightarrow [Y1 \Rightarrow 1$
- $[1] \Rightarrow [X0 \Rightarrow 10$
- $[101] \Rightarrow [10X0 \Rightarrow [1Y10 \Rightarrow [Y110 \Rightarrow 110$
- $[110010] \Rightarrow [11001Y1 \Rightarrow [1100Y11 \Rightarrow [110Y011 \Rightarrow \cdots \Rightarrow [Y110011 \Rightarrow 110011$
- $[111] \Rightarrow [11X0 \Rightarrow [1X00 \Rightarrow [X000 \Rightarrow 1000$
- $[]$ cannot be reduced, it's an illegal input with an illegal output

How about a formal definition?

> An SRS is a tuple (Σ, R) where:
>
> - Σ is the alphabet of symbols
> - $R \subseteq \Sigma^* \times \Sigma^*$ is the rewriting relation.
>
> The one-step derivation relation $\Rightarrow_R \subseteq \Sigma^* \times \Sigma^*$ is defined as: $xuy \Rightarrow_R xvy$ iff uRv.

SRS Computation

An SRS (Σ, R) *computes output* w' *from input* w *iff*

$$[w] \Longrightarrow_R^* w' \wedge \forall x\, y\, z.\, w' = xyz \supset \nexists u.\, (y, u) \in R$$

We can make a SRS be a recognizer, too. Here is an SRS that determines whether or not its input is in the form $a^n b^n c^n$, for $n \geq 0$:

ab	$\rightarrow$	X
Xb	$\rightarrow$	bX
Xc	$\rightarrow$	ε
$[]$	$\rightarrow$	$\boxed{\text{YES}}$

SRS Acceptance

An SRS (Σ, R) *accepts* w *iff* $[w] \Longrightarrow_R^* \boxed{\text{YES}}$

***Exercise**: Show how* $aaabbbccc$ *is accepted. Show how abbbca is rejected.*

***Exercise**: Give a Turing Machine to decide this language.*

Register Machines

Now let's move from tapes to registers.

A **register machine** has a number of registers $r_0, r_1, r_2, \ldots$, each holding a single (unbounded) natural number. The machine's control is a program, defined as a list of (labeled) instructions. The instructions are normally executed in sequence, but some instructions can cause a "jump" to an arbitrary instruction. The instruction set varies from model to model (of which there exist an infinite number of possibilities). Here's an example of a register machine with a very small instruction set:

Instruction	Description
LOAD n, r_k	$r_k := n$
COPY r_1, r_j	$r_j := [r_i]$
ADD r_i, r_j, r_k	$r_k := [r_i] + [r_j]$
SUB r_i, r_j, r_k	$r_k := [r_i] - [r_j]$
... and so on for MUL, DIV, POW, AND, OR, LT, LE, EQ, NE, GE, GT, SHL, SHR	
NEG r_i, r_j	$r_j := -[r_i]$
NOT r_i, r_j	$r_j := \neg[r_i]$

Can also include JUMP, JZ, JE, READ, WRITE, HALT from above

Register machines are rather important, as they are pretty much the basis for most "real" machines today. We'll discuss them in more detail later.

Lambda Calculus Interpreters

Any lambda calculus function is essentially a program. We tend to think of the λ-calculus as a formalism for modeling computation (and computable functions) in general, but one could, in theory, build hardware to carry out the operations of the calculus, so it's fun to consider it along with more classical automata.

An early one was the SECD Machine.

> ***Exercise**: Study this machine.*

μ-Recursive Functions

The μ-Recursive Functions, also known as the **General Recursive Functions**, or the **Partial Recursive Functions**, are those functions defined by a small number of rules. They allow computations in "bigger steps" than the λ-Calculus.

And More!

Yes, people have created probabilistic automata and quantum automata, too.

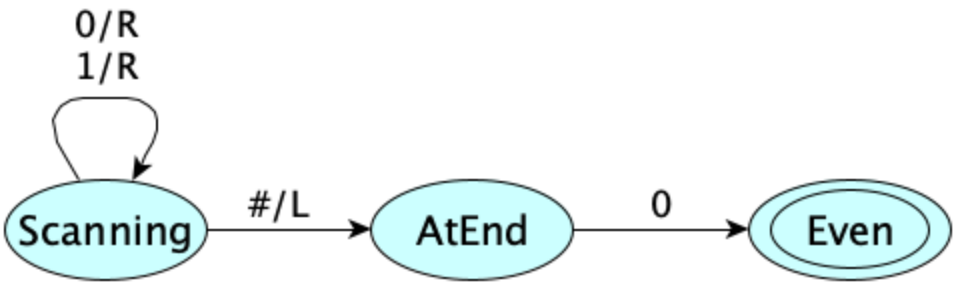
Connections To Formal Languages

A recognizer automata actually defines a formal language! That is, the language of machine M is:

$$L(M) = \{w \mid M \text{ accepts } w\}$$

Nice! This means a formal language can be defined by a grammar or by a machine. This is philosophically interesting, if not profound. It feels like there is a kind of symmetry going here.

How about an example, to understand the important concepts here, and to get the vocabulary right? Let M be the Turing Machine given by the following state diagram:



When run on any non-blank input of binary digits, the machine will end in the accept state iff the input ends with a 0. Otherwise it will get stuck. Therefore we say:

- M **accepts the string** 110100100
- M **recognizes the language** $\{w \in \{0,1\}^* \mid w \text{ is an even binary integer}\}$
- M **accepts the language** $\{w \in \{0,1\}^* \mid w \text{ is an even binary integer}\}$
- M **decides the language** $\{w \in \{0,1\}^* \mid w \text{ is an even binary integer}\}$
- $L(M) = \{w \in \{0,1\}^* \mid w \text{ is an even binary integer}\}$

> ***Accepting vs. Deciding***
>
> *In tne example above, the machine M did indeed both accept and decide the language of even binary integers. However, there do exist languages that are recognizable by a TM but not decidable by any TM. The difference between accepting and deciding is part of computation theory, and we'll study this later.*

Connections To Computability Theory

We can characterize different automata by the classes of functions they can compute, or equivalently, the languages they can recognize. For example Turing Machines can recognize a much larger class of languages than can Type-3 Machines.

So we can rank automata in increasing "power." In the following table, the class of languages/problems in each row is a proper subset of the following row:

Automata	Problems that can be solved / Languages that can recognized	Example
Loop-free FA	Finite Languages. Languages with a finite number of strings, you can basically just list them all out. Here the FA is simply a directed-acyclic graph.	$\{a, b\}$
FA (Type-3 Machine)	Regular Languages. You can recognize these without any external memory whatsoever. And you can process them in linear time.	a^*
Deterministic PDA (DPDA)	Deterministic Context-Free Languages. An interesting subset of the context-free languages.	$a^n b^n$
Nondeterministic PDA (NPDA)	Context-Free Languages. It's quite interesting, and not really too hard to prove, that the NPDAs recognize exactly the class of CFLs, which are the languages generated by a Context-Free (Type-2) Chomsky Grammar. This is a fairly natural class of languages.	ww^R
LBA	Type-1 Languages. Bounding the tape of a TM reduces the computing power significantly!	$a^n b^n c^n$
TM that always halts	Decidable Languages. Also known as the Recursive Languages, **R**.	LBA_{diag}
TM	Type-0 Languages. Recursively Enumerable Languages, **RE**. For languages that are in RE but not in R, a TM can be found that will accept (say YES) every string in the language, but will not be able to halt and say NO for all strings not in the language.	H
—	Finitely Describable Languages. Any language that can be finitely described, whether it can be computationally recognized or not.	$\overline{H}$

Can any automata compute a larger class of functions than a Turing Machine? We don't think so. In fact, the following are all equal in computing power:

- Turing Machines
- Deterministic TMs

- One-way infinite tape TMs
- Multi-tape TMs
- Multi-dimensional TMs (e.g., can move N, E, S, W)
- Queue Automata
- NPDAs with two stacks
- Register Machines
- RAMs
- RASPs
- The μ-recursive functions
- Post Systems
- Phrase-Structure (Type-0) Grammars
- The Lambda Calculus

The fact that these all are essentially the same thing, and all attempts at making them more powerful don't actually increase their power, led to the **Church-Turing Thesis**, namely, "The Turing Machine captures exactly we intuitively think of as computable."

> ***Exercise**: Read about the Chruch-Turing Thesis at the Stanford Encyclopedia of Philosophy and at Wikipedia.*

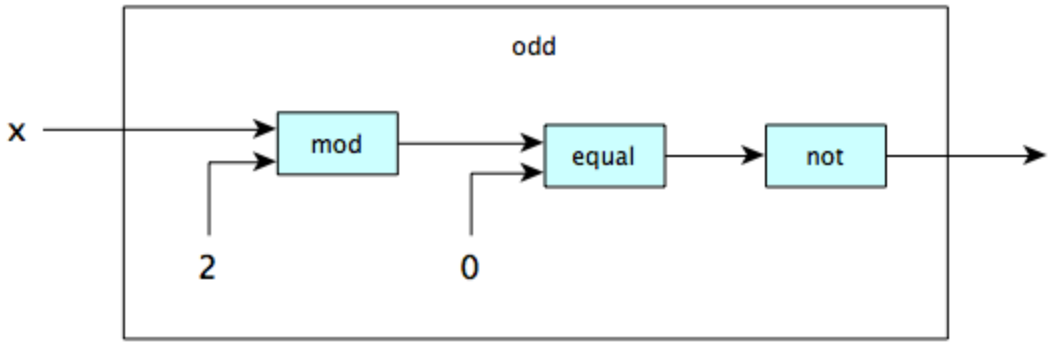
Applications

One of the reasons we have theories is to apply them. (Remember, a theory is an organized body of knowledge with explanatory and predictive powers.) What are some applications of automata?

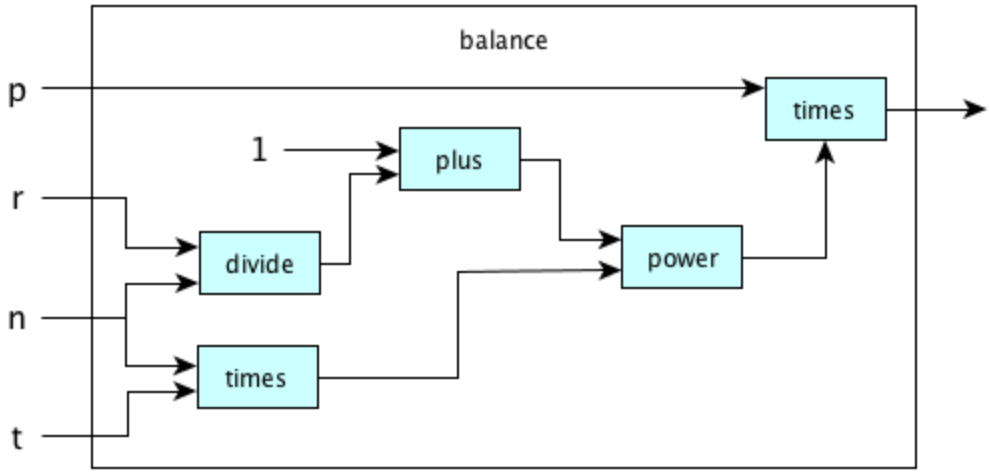
Type-3 machines are used in Lexical Analysis, (extracting "words" from a string of characters), an important operation in text processing, and especially in compiler writing.

Type-2 machines are used in Parsing, uncovering the structure of a string of symbols, another hugely important and common operation.

We can see the relationship between automata and functions, through pictures like:



ant this one:



These pictures are good for humans to build a conceptual understanding of programming by combining functions.

Where to Lean More

Check out the following sources for more information:

- Wikipedia article on Automata Theory
- A Short Introduction by some folks at Stanford in 2004
- The Sipser Theory of Computation Book

Summary

We've covered:

- ✅ What automata theory is
- ✅ Some history of automata theory
- ✅ Various types of automata
- ✅ Relationship of automata theory with formal language theory
- ✅ Relationship of automata theory to computability theory

Turing Machines

There was a time, not too long ago, when people had not yet precisely defined what computation was. Alan Turing took on this task. Here is what he came up with.

CONTENTS

The Backstory • The Basic Idea • Thirteen Examples • Formal Definition • More Examples • Variations on the Turing Machine • An Online Simulator • The Universal Machine • Busy Beavers • Summary

The Backstory

History helps a lot! Why do we have Turing Machines? Why do we care? Why are we better knowing about Turing Machines than not knowing them?

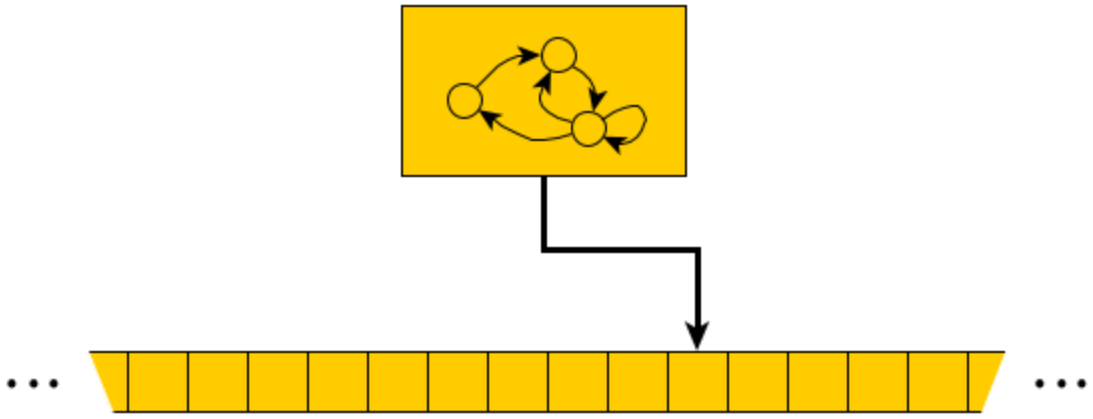
Hear the answers in class. We'll go through 19th century advances in logic, the formalists versus the intuitionists, Hilbert's Challenges, the need to formalize computation, Gödel's and Church's approaches, and how Turing's view of computation was the most mechanistic (as opposed to logical or mathematical) and influenced by human computers of the day, and how he (Turing) translated all these ideas into simple mathematical constructs that in some real sense....nailed it!

The Basic Idea

Turing looked at computers of his day and asked "What are these people doing?" He thought about it and realized these folks would write something down, think for a bit, write something else or erase something, think some more, write or erase, and so on. They would move from mental state to mental state as they worked, deciding what to do next based on what mental state they were in and what was currently written.

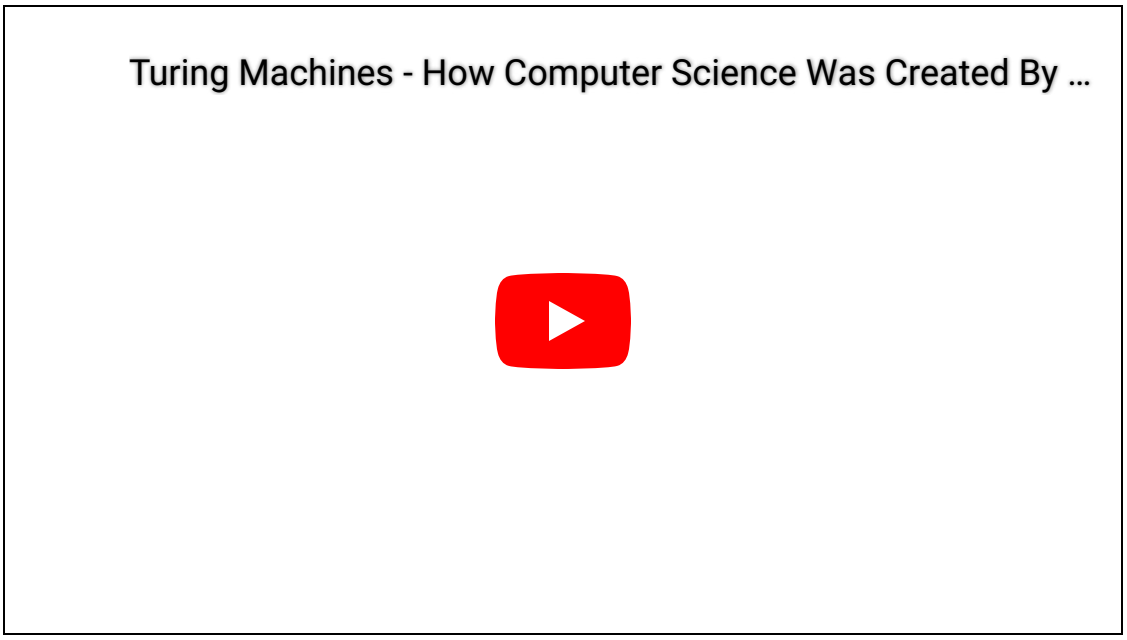
He realized he could capture this process formally and keep the model very simple: instead of a two-dimensional page, a single one-dimensional tape would do, and instead of randomly jumping around, it would be okay to move one cell at a time. Today we picture the machines like this:





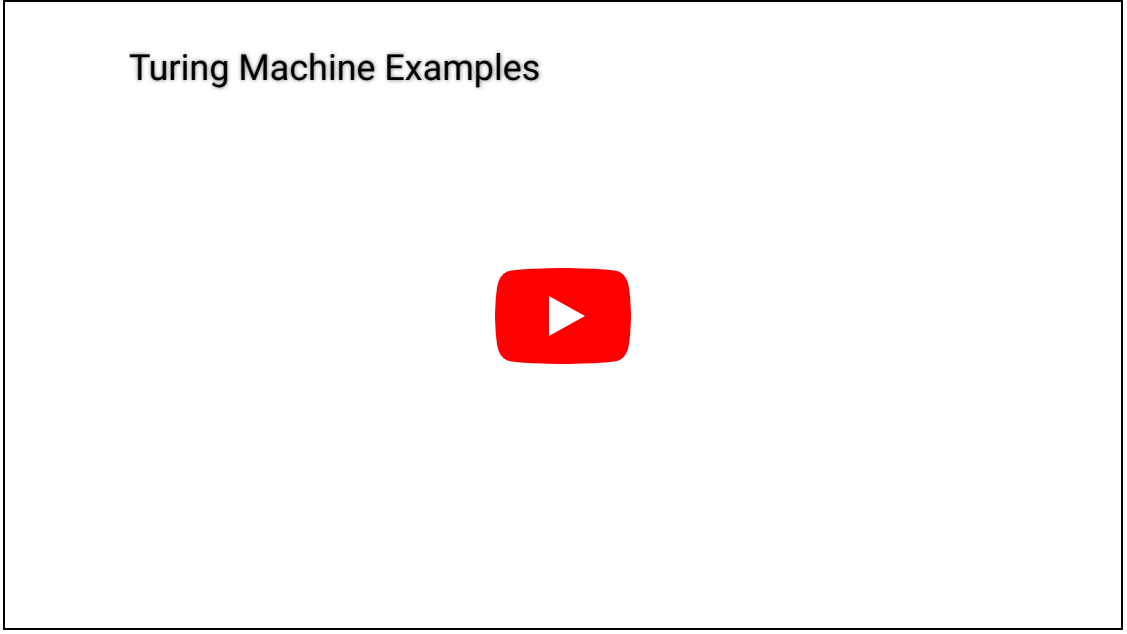
SIngle infinite tape for input and output

Jade's video, again:



Thirteen Examples

Let's see some examples before we get formal.



Formal Definition

Y'all set for a formal definition? Here is the definition for a transducing TM:

> A **Turing Machine** is a tuple $(Q, \Sigma, \Gamma, b, R, S)$ where:
>
> - Q is a finite set of states
> - Σ is the alphabet of symbols that comprise our inputs and outputs
> - Γ is the symbols allowed to appear on the tape
> - $b \in \Gamma - \Sigma$ is the blank symbol
> - $R \subseteq (Q \times \Gamma) \times (\Gamma \times \{\mathsf{L}, \mathsf{R}\} \times Q)$ says what can be done in a given state looking at a given symbol
> - $S \in Q$ is the distinguished start state
>
> A **configuration** Ψ is a tuple $(u, q, v) \in \Gamma^* \times Q \times \Gamma^*$ describing a machine with uv on the tape preceded by, and followed by, an infinite number of blanks, with the machine looking at the first symbol of v.
>
> The **transition relation** $\Longrightarrow$ on configurations relates configurations with the configurations they give rise to after one **step** of execution. Formally (note $u, v \in \Gamma^*$ and $\sigma, \sigma' \in \Gamma$):
>
> $$\begin{aligned} \Longrightarrow_R \quad &= \quad \{((u, q, \sigma v), (u\sigma', q', v)) \mid (q, \sigma)\, R\, (\sigma', \mathsf{R}, q')\} \\ &\cup \quad \{((u\sigma, q, v), (u, q', \sigma' v)) \mid (q, \sigma)\, R\, (\sigma', \mathsf{L}, q')\} \end{aligned}$$

> The machine $(Q, \Sigma, \Gamma, b, R, S)$ **computes** w' from w if $w, w' \in \Sigma^*$ and
>
> $$\exists\, u\, q\, \sigma\, v.\ (\varepsilon, S, w) \Longrightarrow_R^* (u, q, \sigma v) \wedge w' = \mathsf{trim}(u\sigma v) \wedge \nexists \Psi.\ (q, \sigma)\, R\, \Psi$$

What about the recognizer flavor? It's just this:

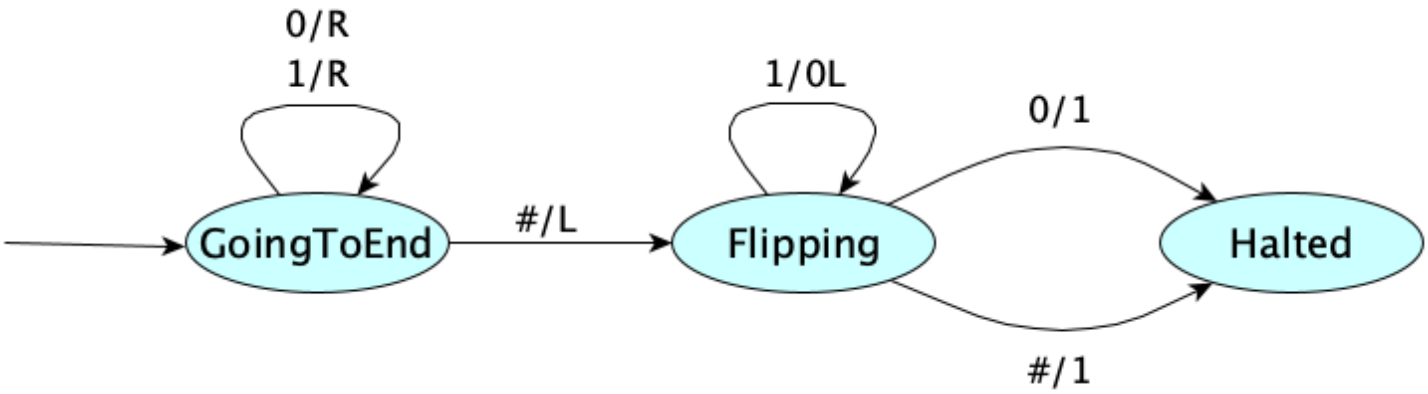
> A **Recognizing Turing Machine** is a tuple $(Q, \Sigma, \Gamma, b, R, S, A)$ where:
>
> - $(Q, \Sigma, \Gamma, b, R, S)$ is as defined above
> - $A \subseteq Q$ is a distinguished subset of **accept states**
> - Configurations and the transition relation are as above
>
> The machine $(Q, \Sigma, \Gamma, b, R, S, A)$ **accepts** the string w if $w \in \Sigma^*$ and
>
> $$\exists u\, q\, \sigma\, v.\ (\varepsilon, S, w) \Longrightarrow_R^* (u, q, \sigma v) \wedge q \in A \wedge \nexists \Psi.\ (q, \sigma)\, R\, \Psi$$
>
> The set of all strings accepted by machine M is said to be $L(M)$, the language **recognized** by M.

***Exercise**: Study the definitions above until you understand everything about them and can regenerate them without looking. Don't just memorize. Understand.*

> ***There is more than one kind of Turing Machine***
>
> *The definition above is the one I like best.*
>
> *In this definition, the transition relation was allowed to be any relation whatsoever. In fact, it is* **non-deterministic***: at each step, the machine may have multiple choices regarding what to next! And the machine runs until it hits a configuration with nowhere to go.*
>
> *We will see variations later, including* **deterministic** *models which require the transition relation to be a function, and a special* HALT *state, or* HALT *action, to be introduced.*

Case Study

Let's use the formal definition to describe a Turing Machine that increments a binary number. Here's the machine as a picture:



Let's identify the parts. Our machine is $(Q, \Sigma, \Gamma, \#, \mathsf{R}, \mathsf{G})$ where:

> $Q = \{\mathsf{G}, \mathsf{F}, \mathsf{H}\}$
> $\Sigma = \{0, 1\}$
> $\Gamma = \{0, 1, \#\}$
> $R = \{$
> $((\mathsf{G}, 0), (0, \mathsf{R}, \mathsf{G})),$
> $((\mathsf{G}, 1), (1, \mathsf{R}, \mathsf{G})),$
> $((\mathsf{G}, \#), (\#, \mathsf{L}, \mathsf{F})),$
> $((\mathsf{F}, 1), (0, \mathsf{L}, \mathsf{F})),$
> $((\mathsf{F}, 0), (1, \mathsf{L}, \mathsf{H})),$
> $((\mathsf{F}, \#), (1, \mathsf{L}, \mathsf{H})),$
> $\}$

We can condense this a bit, using only the rule set, making sure the start symbol appears first (spaces added for clarity):

```
G00RG G11RG G##LF F01LH F#1LH
```

You can come up with a canonical encoding, where states are 0, 1, 2, ...; Σ is $\{0, 1\}$; the symbols in $\Gamma - \Sigma$ are 2, 3, 4, ... with the blank being 2; Left is 0 and Right is 1:

```
00010 01110 02201 10102 12102
```

More Examples

If the 13 examples in the video were not enough, we have more.

Binary Decrement

TODO

Reverse a String

TODO

Binary Addition

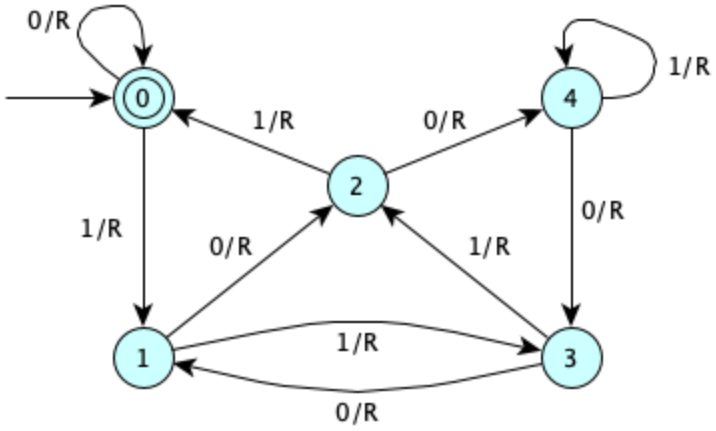
TODO

Parity

TODO

Is Divisible by 5?

Take the empty string to be 0. Read the binary string from left to right. Whenever you see a zero, your current number goes from n to $2n$. Whenever your see a 1, your number increases to $2n + 1$. Keep track of the current number modulo 5. You accept if you finish with a number whose value mod 5 is 0.



Even More Examples

Want even more examples? Check out this article with six interesting examples, including:

- Binary addition
- Converting binary to unary
- Converting unary to binary
- Doubling the length of a sequence
- The 4-state busy beaver
- Palindrome detection

Variations on the Turing Machine

Let's mix things up a bit.

Determinism

We've defined our machines with a transition relation that could have multiple outgoing transitions from a state for the same configuration. But what if we required the machine to have, for every state, one and only one transition? That is, instead of:

$$R \subseteq (Q \times \Gamma) \times (\Gamma \times \{L, R\} \times Q)$$

we required R to be a function? Doing so would require us to expand the codomain of the function so that the machine could stop. We could add a **halting action**:

$$R : (Q \times \Gamma) \to (\Gamma \times \{L, R\} \times Q) \cup \{\text{HALT}\}$$

This says: if the machine is in some state $\sigma \in Q$, and currently looking at a symbol $\sigma \in \Gamma$, then either:

1. Replace the symbol with a new symbol σ', move one cell, and enter state $q' \in Q$, OR
2. Halt

A slightly different definition of a deterministic Turing Machine treats "halting" as a kind of state, a **halting pseudo-state**:

$$R : (Q \times \Gamma) \to (\Gamma \times \{L, R\} \times (Q \cup \{\text{HALT}))\}$$

This says: if the machine is in some state $\sigma \in Q$, and currently looking at a symbol $\sigma \in \Gamma$, then:

1. Replace the symbol with a new symbol σ', move one cell, THEN
2. Either enter state $q' \in Q$ or Halt

> ***Exercise**: Study and appreciate the subtle differences between these definitions. Do you feel your mathematical maturity rising?*

CLASSWORK

How many n-state deterministic TMs are there?

Recall from your discrete math class that the number of functions over finite domain D and finite range R is $|R|^{|D|}$ — you did pay attention in that class, right? Haha, it's okay, I had forgotten that too.

We will show that, for the halting-pseudo-state version, the domain has size $2n$ and the range has size $4(n+1)$.

▶ Hint

Later we will see why this is interesting.

The definition is a little clunkier, but it's kind of nice. Interestingly, this simplification does not change the computing power of the model.

Stationary "Moves"

Instead of having to go left or right at each step, you can also provide to option to stay put. No change to computing power.

$$R \subseteq (Q \times \Gamma) \times (\Gamma \times \{\mathsf{L}, \mathsf{R}, \mathsf{X}\} \times Q)$$

Here X means stay on the same cell.

ε-Moves

In the base model, the machine looks at the current symbol and decides what to do next. We can let it just decide to jump to another state without examining the current symbol. Again, no change in computing power.

$$R \subseteq (Q \times (\Gamma \cup \{\varepsilon\})) \times (\Gamma \times \{\mathsf{L}, \mathsf{R}\} \times Q)$$

Multidimensional Tapes

How about, instead of a tape that is infinite only in the left and right directions, we make a two dimensional tape that can go up and down also? Or a 3-D tape? Or make a machine whose tape is any finite number of dimensions? All good. These extensions do not increase the computing power of the model.

One-way Infinite Tape

What if the tape had a left-end, and was infinite only to the right? A little messier to program, perhaps, but no change in computing power.

Multiple Tapes

More than one tape? More convenient. But you guessed it, no change in computing power.

Stack and Queue Tapes

If we restricted our tape memory to act like a queue, we have a Queue Automaton and no change in computing power. But if our tape was stack-like, we've reduced our power: we can no longer compute certain functions (recognize certain languages). But if we had **two stacks** we are back to full power.

Read-Only and Write-Only and Append-Only Tapes

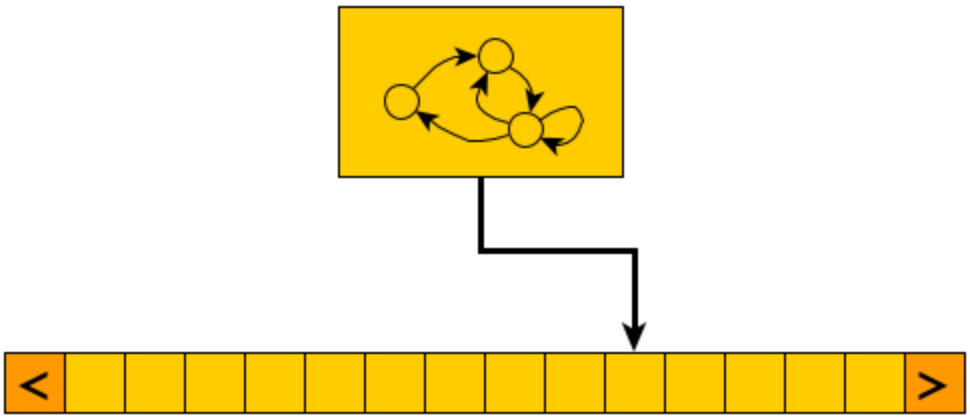
Sometimes we might want to have a bunch of tapes and use certain tapes only in certain ways. Maybe we can only read or only write or only append. Hmmm, tapes are starting to look flexible. We can imagine a tape that is:

- Unbounded on both ends, bounded on one end and unbounded on the other, bounded on both ends
- 1-dimensional, 2-dimensional, ... n-dimensional
- Read-only, Write-only, Append-only

- Stack-like, Queue-like, Move one cell at a time, Take big jumps
- One way (say right-moves only or left moves only) or multi-way

Linear Bounded Automata (LBAs)

What if we bounded the tape on both ends? There's this thing called a **Linear Bounded Automaton** which is a TM with a bounded tape, just big enough to hold the input and two additional special tape symbols, the left-end-marker and the right-end-marker. These symbols can not be overwritten, and you cannot move beyond them. They are super convenient because how else would keep from falling off the ends?



Fixed-size tape bounded to input string plus two cells for endmarkers

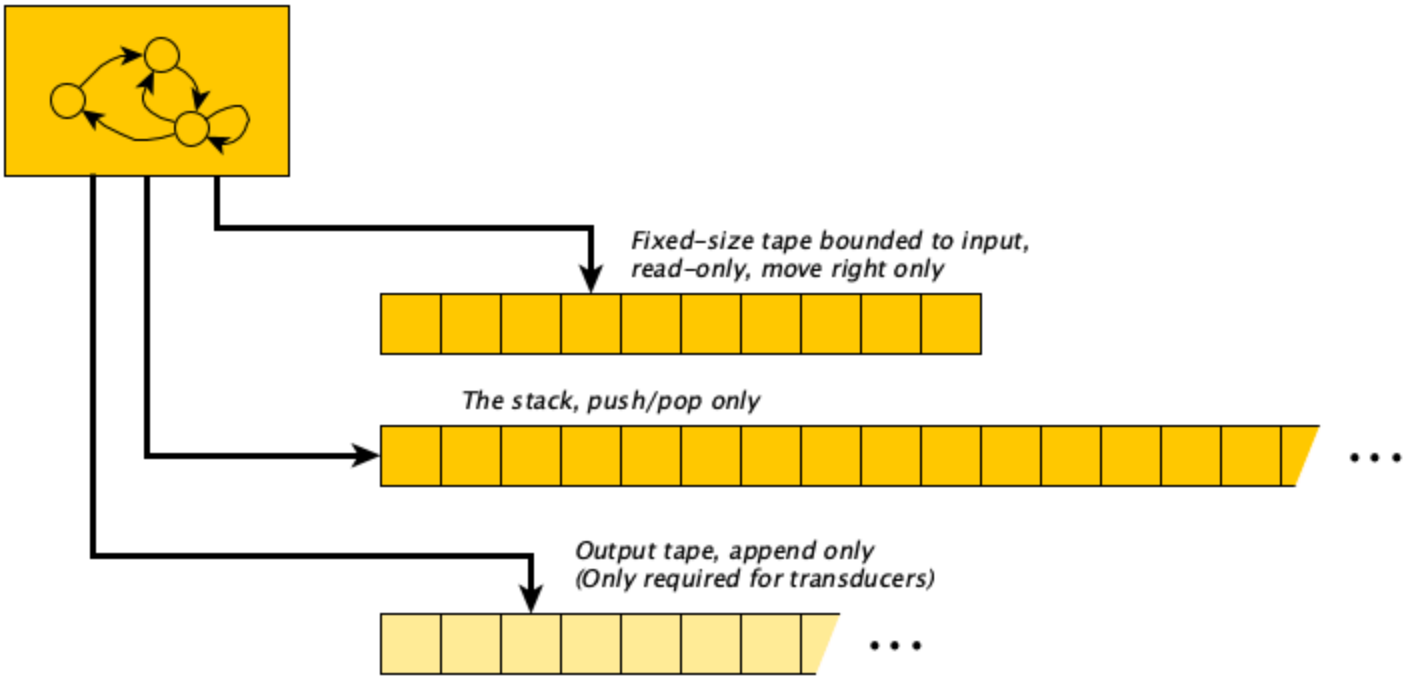
Any change in computing power? It turns out yes. The computing power is reduced.

> ***Exercise**: (Research) Find a language recognizable by a Turing Machine but not by an LBA.*

Pushdown Automata (PDAs)

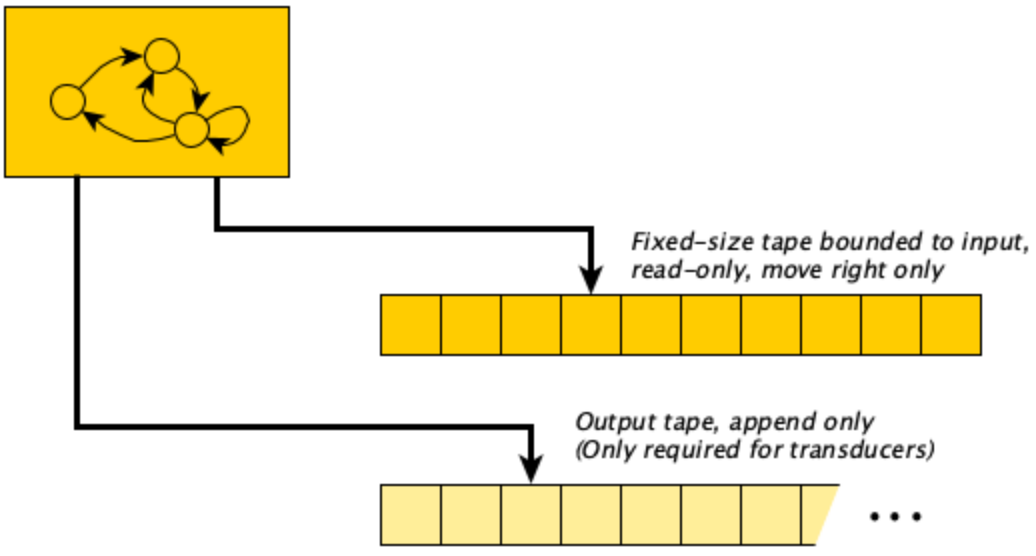
It turns out that the class of context-free languages can be decided exactly by a machine with two tapes: (1) a bounded, read-only, right-move only table holding exactly the input, and (2) a stack-like, one-way infinite tape for whatever you need. This kind of machine is called a push-down automaton (PDA). It is useful because stacks are super efficient, $\Theta(1)$ for pushing and popping. The PDA runs until there is no transition to make, or until the machine exhausts its input.

If you want a PDA transducer, add an output tape.



Finite Automata (FAs)

What if we had no memory at all? We'd have something called a **Finite Automaton**.



A lot of computations fall into this category.

An Online Simulator

When writing a Turing machine to carry out a computation or decide a question, don't just guess! Use a real online simulator to check your work!

> ***Exercise**: Write your own simulator.*

The Universal Machine

The most brilliant idea ever.

OMG.

Check this out. Remember that Turing Machine we saw above to increment a binary number? Remember how it could be encoded as a string:

```
0001001110022011010212102
```

Every Turing Machine can be represented as a string. Turing Machines take strings as input. Wouldn't it be cool if we could make a single TM that could read the description of a machine M, along with some other input w, and simulate M running on w?

Turing showed this was possible.

A machine that that do this is called a **Universal Turing Machine**.

The existence of a UTM shows that general purpose computers are possible. Indeed Turing's work predated real general purpose computers.

To see how Turing found the Universal Machine, start at Wikipedia.

Busy Beavers

An **n-state busy beaver** is a deterministic n-state, halting, Turing Machine with $\Sigma = \{1\}$ and $\Gamma = \{b, 1\}$ that writes the largest number of 1s on an initially blank tape over the set of all such n-state, halting Turing Machines, using the *second* definition of deterministic Turing Machine above (the kind that is allowed to write a symbol before halting).

Why on Earth would be care about such things? Let's find out. First, let's see what we are up against. Previously we saw that the number of deterministic n-state Turing Machines is $(4(n+1))^{2n}$. Hey that's a lot of Turing Machines! For fun, here are the first few values:

n	$(4(n+1))^{2n}$
0	1
1	64
2	20736
3	16777216
4	25600000000
5	63403380965376
6	232218265089212416
7	1180591620717411303424
8	7958661109946400884391936
9	68719476736000000000000000000

For a given n, some of these machines will halt, and some will not. Now we can go through all the halting ones (haha) and count the number of 1s they print out. The maximum number we find is $\Sigma(n)$. Let's see what these values are:

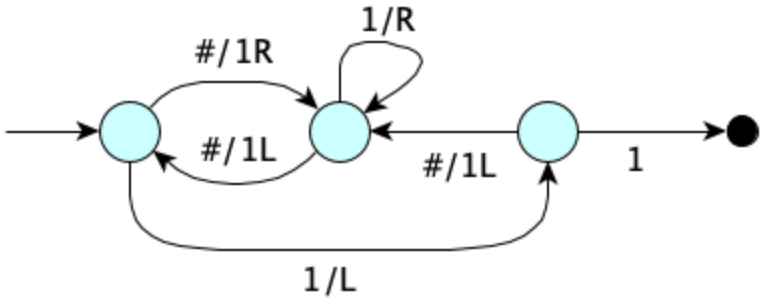
n	$\Sigma(n)$
0	0
1	1
2	4
3	6
4	13
5	at least 4098
6	greater than 3.515×10^{18267}
7	greater than $10^{10^{10^{10^{18705353}}}}$

Woah.

It seems amazing that we *know* what $\Sigma(4)$ even is! Knowing this means that for each of the 25.6 BILLION 4-state, 2-symbol Turing Machines, someone figured out whether it halts or not, and if it halts, what it outputs. Finding $\Sigma(5)$ will be harder: there are over 63.4 TRILLION machines to investigate.

That's a lot of machines to go through. And it's not easy to figure out whether a long-running machine will eventually halt or not. Such a machine might take years or centuries to finish, so we can't just run it and see. We have to painstakingly analyze each machine, and we have to sometimes be really clever. And if we know a long running machine will halt, we need a clever way to figure out how many 1s it will output, because this number might be too big to actually store. Can you see now why people that try to find these values are called **Busy Beaver Hunters**?

Here is a 3-state Busy Beaver you can try out:



You can read more about Busy Beavers at OIS and at Wikipedia.

> ***More Busy Beaver Fun To Follow***
>
> *When we study computability theory, we'll see that Σ is not a computable function. Although it has a well-defined finite description, no computing device can compute it. In fact, it grows faster than any computable function.*
>
> *Imagine that.*

Impatient? Here is a very nice short paper with formal proofs that Σ, and its friend S (function for the maximum number of *steps* a 2-symbol, n-state DTM takes), are uncomputable.

Here's more on Busy Beavers:

The Boundary of Computation



Summary

We've covered:

- ✅ Historical motivation
- ✅ How Turing viewed computation
- ✅ Lots of Examples
- ✅ Formal Definition
- ✅ Variations
- ✅ Universality
- ✅ Busy Beavers

Introduction to Parsing Theory

Compilers need to parse. A parser is just code, so to build one, we could just wing it. But a theory helps us understand parsing at a deep level, enabling us to write better, faster, and provable correct parsers. Let's learn!

CONTENTS

What is Parsing Again? • Lexical Analysis • Approaches to Parsing • Recursive Descent • Table-Based Parsing • Parsing Expression Grammars • Real-World Issues • Resources • Summary

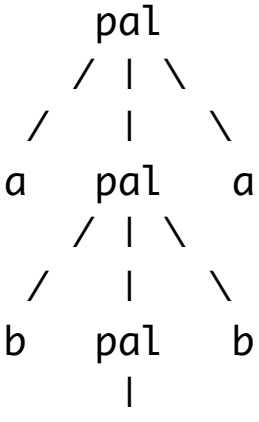
What is Parsing Again?

Parsing is the process of uncovering the derivation, or derivation tree, of a string from a grammar.

For example, given the grammar:

pal ⟶ ("a" | "b" | "a" *pal* "a" | "b" *pal* "b")?

we **parse** the string ***abbba*** into the derivation tree:



|
b

In a real programming language, we generally don't make a derivation tree for each character in the source program. Instead, we first group characters into **tokens** and we consider the tokens as the symbols.

TODO

The pre-processing of the source program via tokenization is so common and so natural and so important that there is a kind of separate (simpler) theory and practice that deals with this, called **lexical analysis.** Let's briefly talk about that first before getting into the richer theory of parsing.

Lexical Analysis

Tokenization, or lexical analysis, is the process of forming tokens out of the source code character stream. You basically look at the character stream character-by-character, skipping whitespace and comments, and collecting characters into tokens. Sometimes you might need to "look ahead" a little, or "back up" as you go through the character stream.

Generally, each token will have a type, a lexeme, and line and column position. Types of tokens include:

- keyword
- identifier
- numeric literal
- string literal
- punctuation

The **lexical grammar** is the part of the language specification that shows how tokens are formed from characters. Negative lookahead is common! An example might be:

TODO

Sometimes people find it helpful to visualize how a **lexer** works via a Finite Automata:

TODO

Writing a lexer by hand is possible. For the simple lexical grammar shown above, a hand-crafted lexer might be:

TODO

In general, lexers are concerned with issues such as:

- Case sensitivity (or insensitivity)
- Whether or not blanks are significant
- Whether or not newlines are significant
- Whether comments can be nested

Errors that might occur during scanning, called **lexical errors** include:

- Encountering characters that are not in the language's alphabet
- Too many characters in a word or line (yes, such languages do exist!)
- An unclosed character or string literal
- An end of file within a comment

Once you have a lexer written and deployed, you can now work entirely with token streams. Parsing now works entirely at the level of the phrase grammar. The parser's job is to uncover the derivation tree, or parse tree, whose leaves are now tokens, not characters.

Approaches to Parsing

A language's syntax is normally defined with a CFG (or the equally powerful BNF, EBNF, Syntax Diagram), or a PEG. A compiler uses the grammar to **parse** the token sequence, that is, *to determine if the token string can be generated by the grammar, and produce the syntax tree for the token sequence*.
People have been studying parsing for decades! Here are some things they figured out:

- Context-sensitivity is too hard. Separate it out. Just focus on the context free parts of the language in the parser.
- There's a desire for grammars for real programming languages to be **unambiguous**.
- Parsers should be fast! There is a theorem that says every CFG can be parsed in $\Theta(n^3)$ time. That is too slow. Perhaps we should look for restricted CFGs that can be parsed more efficiently.

- There is! People have figured out and defined the classes of LL grammars and LR grammars! These are restricted CFGs that can be parsed in $\Theta(n)$ time. Slay!
- Massive amounts of theory have been generated around parsing techniques for LL and LR grammars, since the mid-20th century. LL mechanisms are easy but the class of LL languages is much smaller than LR. But LR methods are really tough to follow and not meant for human understanding.
- PEGs hit the scene in 2004 and are awesome! They are a little different from CFGs. They can be parsed in linear time with a packrat parser, but at the cost of a lot of additional space. Ohm adds some power to PEGs (left recursion) so you *might* encounter super-linear parse times.

That's some good background info to start a study of parsing. But wait, first, what do LL and LR mean?

$LL(k)$ Grammar

A grammar that can be used to drive a leftmost derivation by reading the input left-to-right, examining only the next k symbols, and never backing up.

$LR(k)$ Grammar

A grammar that can be used to drive a rightmost derivation by reading the input left-to-right, examining only the next k symbols, and never backing up.

Um, are there more practical descriptions of these language classes? Pretty much, yeah. Try these:

$LL(k)$

Locate all the "choice points" in the grammar (these are easy to find in the equivalent syntax diagram). Determine the maximum number of tokens, k, required to know exactly which path to take over all choice points. If k is finite, the grammar is LL(k).

$LR(k)$

The set of LR languages is exactly equivalent to the set of languages recognizable by a DPDA. LR languages are exactly the DCFLs. An $LR(k)$ grammar is a CFG for a language that is a DCFL.

An LL and LR Fact Sheet:

LL Grammars	LR Grammars
• Subset of CFGs • Never ambiguous • Parsable in Linear Time	• Subset of CFGs • Never ambiguous • Parsable in Linear Time

• Left-to-right, Leftmost derivation • Top-down • Predictive • Expand-Match • Parsers can be hand-made • Can give nice error messages • Can not handle left recursion • Less "powerful" than LR	• Left-to-right, Rightmost derivation • Bottom-up • Not Predictive • Shift-Reduce • Nearly impossible to write parsers by hand • Must work hard to give good errors • Left recursion is desirable! • More "powerful" than LL

Wow! That was a lot. Details are coming. Read on.

> ***Exercise**: Why is it that a grammar with left-recursion can not be LL(k) for any k?*

> ***Exercise**: For more on LL and LR see this BNF article with a great little section on parsing.*

We've made syntax diagrams to describe all the legal programs in the language, but how to we follow the diagrams to "match" or "parse" a candidate program? That is, given a program (a candidate string), how, exactly, do we figure out how it is matched by, or generated by, the syntax diagrams? How do we **parse**?

There have been hundreds of books and thousands of papers on the topic of parsing over the years. Here we just need to highlight a few things.

Hand-Crafted Parsers

Because a parser is just a program that turns source code into a parse tree, you can just write on by hand. The most common kind of hand-crafted parser uses the technique of recursive descent, which you can read about at Wikipedia. You kind of walk through each of the syntax diagrams.

CLASSWORK

I've written a recursive descent parser as part of an Astro compiler. We'll take a very brief look at it in class.

Parser Generators

A recursive descent parser generally requires a highly restricted syntax, one that is line with a class of languages known as the $LL(k)$ languages. These languages are a proper subset of the $LR(k)$ languages (also known as the deterministic context-free languages), which are used in popular parser generators such as YACC, Bison, and ANTLR, among others. While these systems are quite popular and famous, we won't be covering them here. Do read about them, though.

We will see $LL(k)$ and $LR(k)$ later in the course, though.

Analytic Grammars

Another approach is to use a specialized kind of grammar designed specifically for parsing. So instead of using traditional **generative grammars**, covered previously in our notes on Language Theory (which specify a language by giving a set of rules to **generate** all and only those strings in a language), we can use an **analytic grammar**, in which the rules specify how to **match** a candidate string.

The biggest difference between generative and analytic grammars is in the handling of alternatives. In a generative grammar G, $L(G)$ is contains all strings that *can* be generated by applying rules whenever possible. For example, the rule $S \rightarrow A \mid B$ means you can replace an S with an A or a B, so you can generate both. So the grammar:

```
s ⟶ "a" | "ab"
```

describes the language $\{a, ab\}$.

In an analytic grammar, the rules are deterministic; you don't get to pick one or the other. So $A \ / \ B$ means: "Try to match A. If you can, the rule matches and you are done. Otherwise (and only otherwise), try to match B". So:

```
s ⟵ "a" / "ab"
```

recognizes the language $\{a\}$, but:

```
s ⟵ "ab" / "a"
```

recognizes the language $\{a, ab\}$.

Note the tradeoff here. Determinism means there's never ambiguity (a string can only parse one way, if it can at all), which is great! But the ordering of the rules matters, so you have to be careful to

do what you want. Here's another illustration. If we wanted an analytic grammar for Astro, the rule:

$$\mathit{Primary} \longleftarrow \mathit{num} \;/\; \mathit{id} \;/\; \mathit{Call} \;/\; \ldots$$

would be wrong, as it will *never* match a call!

***Exercise**: Why, exactly, must the Call alternative precede the id alternative?*

CLASSWORK

This might be deep, so we'll try a few more examples.

Unlike generative grammars, analytic grammars feature special **lookahead operators**, including the **negative lookahead operator** (~) and the **positive lookahead operator** (&). This enables the definition of non-context-free languages, despite having only one symbol on the left-hand side! Here's an analytic grammar for the non-context-free language $\{a^n b^n c^n \mid n \geq 0\}$:

$$\begin{array}{lcl} s & \longleftarrow & \texttt{\&}\,(x\ \texttt{"c"})\ \texttt{"a"}^{+}\ y \\ x & \longleftarrow & \texttt{"a"}\ x?\ \texttt{"b"} \\ y & \longleftarrow & \texttt{"b"}\ y?\ \texttt{"c"} \end{array}$$

Parsing Expression Grammars (PEGs)

There are various formalisms of analytic grammars (Wikipedia identifies several); the most popular is the Parsing Expression Grammar (PEG). To read a PEG description, pronounce the operators as follows:

- `A B` : match A then match B
- `A / B` : match A. If it matched, success. Otherwise, match B
- `A?` : match A zero or one times
- `A*` : match A zero or more times
- `A+` : match A one or more times
- `&A` : match A without consuming it (but it must match)
- `~A` : match A without consuming it. If A matches then ~A fails. If A fails ot match, then ~A succeeds
- `end` : matches the end of the string
- Quoted text: successively match each symbol exactly

Note how PEGs use `/` instead of `|`, perhaps to emphasize the deterministic nature of matching.

Strictly speaking, it should be impossible in a PEG to have **left-recursive rules** like:

Exp ⟵ ***Exp*** **"+"** ***Term*** / ***Term***

given that "in order to match an Exp, you much first try to match an Exp." It's sounds circular. In fact it is, and traditionally, PEG implementations suggested using the rule:

Exp ⟵ ***Term*** **("+"** ***Term*****)***

instead. However, it turns out a proper parsing technique (using a **packrat parser**) can deal with it! It's not easy, but you can read the original paper by Alex Warth et al or this nice and detailed article by Guido van Rossum to see how it's done.

> ***Check your understanding***
>
> *Analytic grammars:*
>
> - ✅ *Are designed for parsing (recognition) rather than generation of languages.*
> - ✅ *Are written with only a single variable on the left hand side of each rule.*
> - ✅ *Use ⟵ rather than ⟶ to define rules.*
> - ✅ *Execute alternatives left-to-right and once you get a match, you keep it, even if it leads to a dead end.*
> - ✅ *Have powerful lookahead operators, thanks to deterministic execution.*
> - ✅ *Can handle left-recursive rules.*

Try this exercise before moving on:

***Exercise**: Show that the following generative grammar for assignment statements and if-then-else statements in a programming language:*

```
S → "a" | "i" "x" "t" S ("e" S)?
```

is ambiguous, but that the ambiguity can be resolved by rewriting as a PEG:

```
S ← "a" / "i" "x" "t" S "e" S / "i" "x" "t" S
```

How, exactly, is the ambiguity resolved? (Historical note: this is the dangling else problem, a classic in parsing theory.)

Now let's put everything together to come up with a Parsing Expression Grammar for Astro, that (1) properly separates lexical and phrase rules, (2) deals properly with precedence and associativity, (3) embraces left-recursion, and (4) correctly handles reserved words.

Program	⟵	*Statement*$^{+}$
Statement	⟵	*Assignment* / *PrintStmt*
Assignment	⟵	*id* `"="` *Exp* `";"`
PrintStmt	⟵	*print Exp* `";"`
Exp	⟵	*Exp* (`"+"` | `"−"`) *Term* / *Term*
Term	⟵	*Term* (`"*"` | `"/"` | `"%"`) *Factor* / *Factor*
Factor	⟵	*Primary* (`"**"` *Factor*)? / `"−"` *Primary*
Primary	⟵	*num* / *Call* / *id* / `"("` *Exp* `")"`
Call	⟵	*id* `"("` (*Exp* (`","` *Exp*)*)? `")"`
num	⟵	*digit*$^{+}$ (`"."` *digit*$^{+}$)? ((`"E"`|`"e"`) (`"+"`|`"−"`)? *digit*$^{+}$)?
print	⟵	`"print"` ¬*idchar*
id	⟵	¬*print letter idchar**
idchar	⟵	*letter*| *digit*| `"_"`
space	⟵	`" "`|`"\t"`|`"\r"`|`"\n"`|`"//"` (¬`"\n"` *any*)*

> ***Lexical and Phrase Variables***
>
> *Ohm also adopts the convention of starting the names of lexical variables with a lowercase letter and phrase variables with an uppercase letter.*

Recursive Descent

Table-Based Parsing

Parsing Expression Grammars

PEGs are designed for direct parsing: there's no need to "extract" any kind of machine. The matching process is built into the grammar rules.

TODO

Must reads:

- Wikipedia's article
- The original paper by Bryan Ford
- Short blog post on CFGs vs PEGs
- Using PEGs Derivatives (Paper that pointed out a huge bug in Ford's original paper)
- 2014 paper relating CFGs and PEGs
- 2020 paper on The computational power of PEGs

Real-World Issues

The language designer will have to take into account any ambiguities in the language description. For example, in C, the expression `(x)-y` hase two different syntactic interpretations: if `x` refers to a type, then its a cast of a negation, otherwise it is a subtraction. You can't tell at this phase.

Errors that occur during parsing, called **syntax errors** arise from malformed token sequences such as:

- `j = 4 * (6 - x;`
- `i = /5`
- `42 = x * 3`

TODO: non-syntax errors

Resources

Summary

We've covered:

- ✅ What parsing is
- ✅ Lexical analysis vs. syntax analysis
- ✅ Various approaches to parsing
- ✅ The method of recursive descent
- ✅ Parsing based on Context-Free Grammars
- ✅ A good deal about Parsing Expression Grammars
- ✅ Where to learn more

Ohm



Language definitions should not be written on paper alone. They should be brought to life. There are many libraries out there that will help you define and process (parse, analyze, generate) languages. Ohm is but one of them, but it's totally worth studying.

CONTENTS

What is Ohm? • Getting Started • Ohm Basics • Official Documentation • Example Grammars • Example Applications • More Examples • Recall Practice • Summary

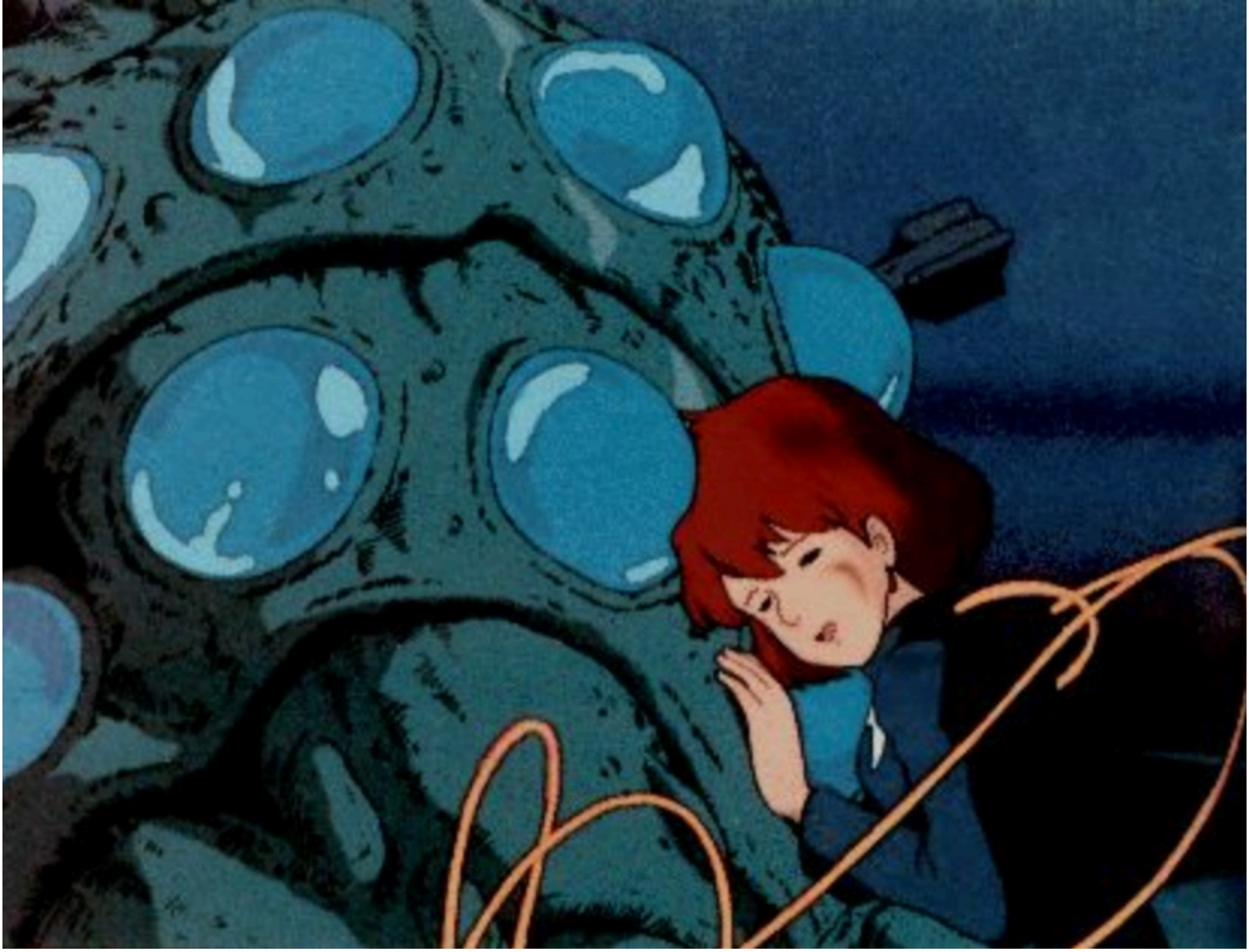
What is Ohm?

Ohm is advertised as a library for building parsers, interpreters, compilers, and more. With it you can quickly generate parsers simply by writing a grammar, then write whatever actions you like to work with the parser output. So with just one grammar you can write:

- A compiler (with multiple targets, even!)
- An interpreter
- A semantics definition
- A syntax highlighter

Yes, you can use Ohm to provide both a formal syntax of a language and a formal semantics (in the form of an interpreter!). Both the syntax definition and the semantics definition are executable in a real sense. Not just math equations on paper!

There's even an interactive tool that enables to to interactively design grammars so that you can prototype new and beautiful programming languages!



Ohm may not generate the fastest parsers but its cleanliness, simplicity, and flexibility make it stand out from the competition.

> ***Ohm Home Page***
>
> *You can find all about Ohm, obtain the source code, read all of its documentation, and find links to tutorials, examples, real-world projects, and the community Discord server, at the home page.*

> ***Ohm Tutorial***
>
> *Pat Dubroy, a co-creator of Ohm, wrote a nice tutorial. It's worth a look as well.*

Getting Started

Here's a short video in which we:

1. Introduce the Ohm Editor and design a grammar for a tiny programming language of arithmetic expressions and print statements
2. Write an interpreter for this language so that one can run programs directly on the command line.

By the way: if you can write a complete interpreter for a language, you can claim to have written a semantics—if you accept that it's fine to say an interpreter *is* the semantics!



Here's the result of the code-along from the video. If you did not watch the video, or don't care to watch it, skip ahead to the Ohm Basics section.

```
import * as ohm from "ohm-js";
import fs from "fs";

const grammar = ohm.grammar(String.raw`PlusMinusLanguage {
  Program   = Statement+
  Statement = "print" "(" Exp ")" ";"
  Exp       = Exp "+" Term                  --plus
            | Exp "-" Term                  --minus
            | Term
  Term      = "(" Exp ")"                   --parens
```

```
            | num
  num       = digit+
}`);

const interpreter = grammar.createSemantics().addOperation("eval", {
  Program(statements) {
    for (const statement of statements.children) {
      statement.eval();
    }
  },
  Statement(_print, _leftParen, expression, _rightParen, _semicolon) {
    console.log(expression.eval());
  },
  Exp_plus(left, _plus, right) {
    return left.eval() + right.eval();
  },
  Exp_minus(left, _plus, right) {
    return left.eval() - right.eval();
  },
  Term_parens(_leftParen, expression, _rightParen) {
    return expression.eval();
  },
  num(digits) {
    return Number(digits.sourceString);
  },
});

const match = grammar.match(fs.readFileSync(process.argv[2]));
if (match.failed()) {
  console.error(match.message);
} else {
  interpreter(match).eval();
}
```

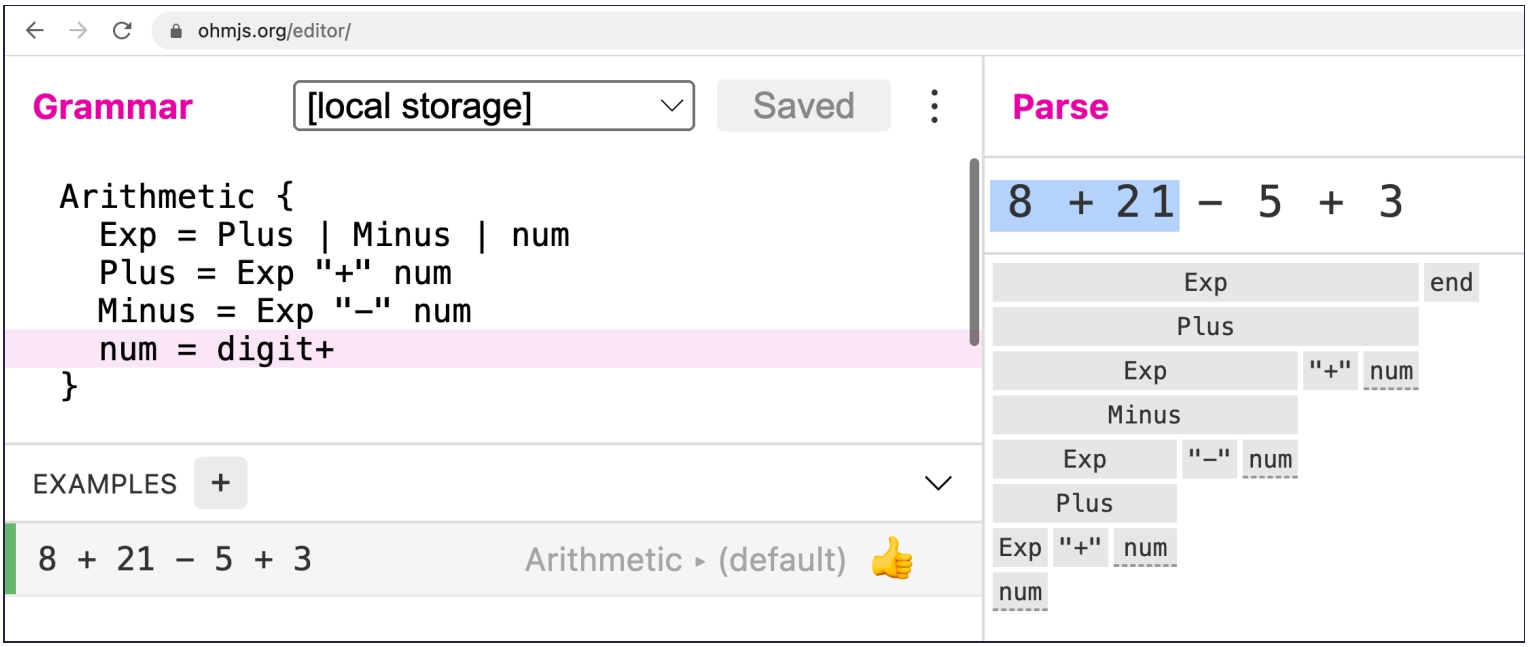
Ohm Basics

The best way to get started is to use the amazing Ohm Editor. While composing your grammar, write and execute lots of text cases. The tests will light up green for success and red for failure as you type. The parse tree will also appear as you type. The immediate feedback cycle is the best way to create.

Watch the video above for details, or if you prefer to save yourself the watching time and just want to read, try out the following famous grammar for starters:

```
Arithmetic {
  Exp   = Plus | Minus | num
  Plus  = Exp "+" num
  Minus = Exp "-" num
  num   = digit+
}
```

Write some test cases and explore the parse trees. Note how the operators are left associative.



Now, let's go through the basics in more detail, and build an actual interpreter!

Setting up a Project

Let's make an interpreter for a real programming language, Astro. Begin as you would to make any Node.js project (with a recent version of Node.js, of course):

```
$ mkdir astro-interpreter
$ cd astro-interpreter
```

```
$ npm init -y
```

Then edit package.json to have a top-level property `"type": "module"`.

Include Ohm in your project:

```
$ npm install ohm-js
```

Check package.json to make sure the Ohm package is there now.

Writing a Grammar

Create the file **astro.ohm** and populate it with the grammar:

astro.ohm

```
Astro {
  Program     = Statement+
  Statement   = id "=" Exp ";"                          --assignment
              | print Exp ";"                           --print
  Exp         = Exp ("+" | "-") Term                    --binary
              | Term
  Term        = Term ("*" | "/" | "%") Factor           --binary
              | Factor
  Factor      = Primary "**" Factor                     --binary
              | "-" Primary                             --negation
              | Primary
  Primary     = id "(" ListOf<Exp, ","> ")"             --call
              | numeral                                 --num
              | id                                      --id
              | "(" Exp ")"                             --parens

  numeral     = digit+ ("." digit+)? (("E" | "e") ("+" | "-")? digit+)?
  print       = "print" ~idchar
  idchar      = letter | digit | "_"
  id          = ~print letter idchar*
  space      += "//" (~"\n" any)*                       --comment
}
```

Design with the Ohm Editor

Especially when working with larger languages, the process of interactively designing with a live tool like the Ohm Editor is pretty much a necessity. Once designed, you'll want to grab your grammar from the editor window and put it into a real software project.

Understanding Parsing Expressions

Ohm grammars consist of a name and a number of parsing expressions. Details are in the syntax reference, which you need to eventually read. But as you may be impatient, here are the Ohm parsing expressions:

Expression	Description
`" ... "`	A literal string. Allows the usual escape chars `\'`, `\"`, `\b`, `\n`, `\r`, `\t`, `\f`, `\\`, `\x` ***hh***, `\u` ***hhhh***, and `\u{` ***h...h*** `}`.
rulename	Just a rule.
rulename < ***params*** >	A rule with parameters.
e *	***e***, zero or more times.
e +	***e***, one or more times.
e ?	***e***, zero or one times.
e_1 e_2	Match e_1 then match e_2.
e_1 | e_2	If e_1 matches, great. Otherwise try to match e_2.
& ***e***	Match ***e*** without consuming it (lookahead).
~ ***e***	Succeed if ***e*** doesn't match, without consuming anything (negative lookahead).
# ***e***	Lexify a syntactic rule.

Understanding Lexical and Syntax Rules

Lexical rules always start with lower case letters and **syntax rules** start with uppercase letters. Syntax rules allow tokens from the class `space` to be implicitly interspersed with other tokens.

Built-In Rules

There are several built-in rules:

- `any` = *any char at all*
- `end` = *the end of the input*
- `space` = `" " | "\x09".."\x0d" | "\ufeff" |` *any char from Unicode category Zs*
- `digit` = `"0".."9"`
- `lower` = *any char from Unicode category Ll*
- `upper` = *any char from Unicode category Lu*
- `unicodeLtmo` = *any char from Unicode category Lt, Lm, or Lo*
- `letter = lower | upper | unicodeLtmo`
- `alnum = letter | digit`
- `hexDigit = digit | "A".."F" | "a".."f"`
- `caseInsensitive<t>` = *any case insensitive version of t*
- `emptyListOf<elem, sep> = /* nothing */`
- `nonemptyListOf<elem, sep> = elem (sep elem)*`
- `listOf<elem, sep> = nonemptyListOf<elem, sep> | emptyListOf<elem, sep>`
- `EmptyListOf<elem, sep> = /* nothing */`
- `NonemptyListOf<elem, sep> = elem (sep elem)*`
- `ListOf<elem, sep> = NonemptyListOf<elem, sep> | EmptyListOf<elem, sep>`

Rule Arities are Important in Ohm

All alternatives for a rule must have the same arity. This is a small price to pay for the massive advantage you get from separating the grammar from the parsing actions that we will see later. But there is a cool short-hand to make it look like alternatives with different arities exist. For example:

```
Exp = Exp "+" Term  --plus
    | Exp "-" Term  --minus
    | Term
```

is an abbreviation for:

```
Exp       = Exp_plus | Exp_minus | Term
Exp_plus  = Exp "+" Term
Exp_minus = Exp "-" Term
```

Grammar Inheritance

Grammars can inherit from other grammars. When inheriting, use:

- `=` to add a new rule
- `:=` to replace the rule from the supergrammar
- `+=` to add an alternative to the rule from the supergrammar

Any grammar you write yourself automatically inherits from the base Ohm grammar, where the built-in rules are defined.

Match Results

Given a grammar g and source code c, g`.match(`c`)` returns a `MatchResult` object, m. If m`.succeeded()` is true, m wraps a parse tree, and you can apply a semantics object to it. If m`.failed()` is true, a detailed error message will be in m`.message` and a smaller error message will be in m`.shortMessage`.

In the Astro interpreter we're building, create the file **astro.js**. This file will house the program. We can get started by populating it with:

```
import * as ohm from "ohm-js"
import fs from "fs"

const astroGrammar = ohm.grammar(fs.readFileSync("astro.ohm"))
const match = astroGrammar.match(process.argv[2])
if (match.failed()) throw new Error(match.message)
console.log("Syntax ok")
```

In your terminal, invoke

```
$ node astro.js 'print(3 + 5.5);'
```

And you should see the response:

```
Syntax ok
```

Applying Semantics

Once we have a parse tree we can process it with what Ohm calls a **semantics object**. You can write a semantics object to do *anything you want* with your parse tree, which is why Ohm is awesome.

> ***Ohm's Superpower***
>
> *In other language processing libraries, like ANTLR and Bison, semantic processing rules are embedded inside the grammar. Ohm clearly separates them. This makes it easy in Ohm to write many sets of semantic actions for a single grammar.*

The thing we are going to do with these notes is write an interpreter.

A semantics object is a collection of operations and attributes. Operations and attributes are applied to each node in the parse tree. Operations and attributes are similar: operations are functions (called every time), and attributes are memoized and called with property syntax. There are default things to do when a semantics object doesn't have operations or attributes for all the rules. There's a lot going on, because Ohm is so powerful. So let's just jump right in to adding an interpreter to our running example.

Here's the complete **astro.js**:

astro.js

```
import * as ohm from "ohm-js"
import fs from "fs"

const astroGrammar = ohm.grammar(fs.readFileSync("astro.ohm"))

const memory = {
  π: { type: "NUM", value: Math.PI, mutable: false },
```

```
  sin: { type: "FUNC", value: Math.sin, paramCount: 1 },
  cos: { type: "FUNC", value: Math.cos, paramCount: 1 },
  sqrt: { type: "FUNC", value: Math.sqrt, paramCount: 1 },
  hypot: { type: "FUNC", value: Math.hypot, paramCount: 2 },
}

function check(condition, message, at) {
  if (!condition) throw new Error(`${at.source.getLineAndColumnMessage()}${messag
}

const evaluator = astroGrammar.createSemantics().addOperation("eval", {
  Program(statements) {
    for (const statement of statements.children) statement.eval()
  },
  Statement_assignment(id, _eq, expression, _semicolon) {
    const entity = memory[id.sourceString]
    check(!entity || entity?.type === "NUM", "Cannot assign", id)
    check(!entity || entity?.mutable, `${id.sourceString} not writable`, id)
    memory[id.sourceString] = { type: "NUM", value: expression.eval(), mutable: t
  },
  Statement_print(_print, expression, _semicolon) {
    console.log(expression.eval())
  },
  Exp_binary(left, op, right) {
    const [x, y] = [left.eval(), right.eval()]
    return op.sourceString == "+" ? x + y : x - y
  },
  Term_binary(left, op, right) {
    const [x, o, y] = [left.eval(), op.sourceString, right.eval()]
    return o == "*" ? x * y : o == "/" ? x / y : x % y
  },
  Factor_binary(left, _op, right) {
    return left.eval() ** right.eval()
  },
  Primary_parens(_leftParen, e, _rightParen) {
    return e.eval()
  },
  Primary_num(num) {
    return Number(num.sourceString)
```

```
  },
  Primary_id(id) {
    const entity = memory[id.sourceString]
    check(entity !== undefined, `${id.sourceString} not defined`, id)
    check(entity?.type === "NUM", `Expected type number`, id)
    return entity.value
  },
  Primary_call(id, _open, exps, _close) {
    const entity = memory[id.sourceString]
    check(entity !== undefined, `${id.sourceString} not defined`, id)
    check(entity?.type === "FUNC", "Function expected", id)
    const args = exps.asIteration().children.map(e => e.eval())
    check(args.length === entity?.paramCount, "Wrong number of arguments", exps)
    return entity.value(...args)
  },
})

try {
  const match = astroGrammar.match(process.argv[2])
  if (match.failed()) throw new Error(match.message)
  evaluator(match).eval()
} catch (e) {
  console.error(`${e}`)
}
```

CLASSWORK

> Lots to explain here!

Running the Interpreter

Let's see the program in action:

```
$ node astro.js 'print(3 + 5.5);'
8.5

$ node astro.js 'x = 5;
y = 3;
print(x ** y * sqrt(100));'
1250
```

```
$ node astro.js '
dozen = 5 + 8 - 1;
print(dozen ** 3 / sqrt(100));
print(sqrt(5.9 + hypot(π, 3.5e-8)));
'
172.8
3.0069241183624493

$ node astro.js '
print(5);
print(x - 10);
y = 100 + x;'
5
Error: Line 3, col 8:
  2 |  print(5);
> 3 |  print(x - 10);
             ^
  4 |  y = 100 + x;
x not defined
```

Note the interpreter accepts the program to interpret as a command line argument. If your Astro program were in a file (say, hello.astro) you can still run the interpreter with:

```
$ cat hello.astro | xargs -0 node astro.js
```

***Exercise**: Change the interpreter so that it interprets a program stored in a file whose name is given as a command line argument.*

Understanding Action Dictionaries

To a grammar, you attach a semantics object. To a semantics object, you can attach operations and attributes. The main property of an operation is an object called an **action dictionary**. The methods of the action dictionary correspond to rules of your grammar, with the left hand side as the method name and the right hand side as the method parameters, which are **parse nodes**. We saw this above.

But we need practice, and more examples. Here's a sample grammar that illustrates the important things to know:

```
Sample {
  S = A B* C+ D? ListOf<E, ","> f
  ...
}
```

Let's set up a semantics object with an operation called `rep`:

```
const semantics = grammar.createSemantics().addOperation("rep", { ... })
```

We'll need a method for `S`. It will have six parameters, each of which are parse nodes. There are three types of parse nodes: (1) **rule application nodes** (for phrase rules), (2) **terminal nodes** (for lexical rules and literal strings), and (3) **iteration nodes** (when `*`, `+`, and `?` are used):

```
S(a, bs, cs, ds, es, f) {
  // a is a *rule application* node
  // bs is an *iteration node* for the (0 or more) bs
  // cs is an *iteration node* for the (1 or more) cs
  // ds is an *iteration node* for the (0 or 1) ds
  // es.asIteration() gets you the *iteration node* for the es
  // f is a *terminal* node
}
```

In the body of `S`:

- Calling `a.rep()` will invoke the method `A`.
- Calling `f.rep()` will invoke the method `f` if it exists; if it does not, it will invoke the special method `_terminal`. By default, an implementation of `_terminal` already exists: it returns `this.sourceString`, but you can override this if you want.
- Calling `bs.rep()` will invoke the method `_iter` on the children of `bs`. The method `_iter` is NOT defined by default! You *could* define it yourself like

  ```
  _iter(...children) {
    return children.map(c => c.rep());
  }
  ```

  but it is much better to write things like `myNode.children.map(c => c.rep())` directly each time you have to process an iteration node, since it makes clear to a reader that you have an iteration!

Don't forget to use `asIteration` on your `ListOf` (and similar) nodes.

Official Documentation

These notes you are reading now are not exhaustive, but they will get you started! There is much more to Ohm. You will need to eventually check out the documentation.

Once you're feeling pretty good, you will find the Syntax Reference, API Reference, and Patterns and Pitfalls most useful.

Time to move to examples.

Example Grammars

Let's see Ohm in use with a few examples of our own. (There are more in the GitHub repo for Ohm.)

CLASSWORK

> We'll be going through these line by line, discussing not only Ohm specifics, but also a bit about language design in general and the intended meaning of each syntactic construct.

Bella

Bella is one of the example languages for this class. Here is its Ohm grammar:

bella.ohm

```
Bella {
  Program   = Statement+
  Statement = let id "=" Exp ";"                        -- vardec
            | function id Params "=" Exp ";"            -- fundec
            | Exp7_id "=" Exp ";"                       -- assign
            | print Exp ";"                             -- print
            | while Exp Block                           -- while
  Params    = "(" ListOf<id, ","> ")"
  Block     = "{" Statement* "}"
```

```
  Exp       = ("-" | "!") Exp7                              -- unary
            | Exp1 "?" Exp1 ":" Exp                         -- ternary
            | Exp1
  Exp1      = Exp1 "||" Exp2                                -- binary
            | Exp2
  Exp2      = Exp2 "&&" Exp3                                -- binary
            | Exp3
  Exp3      = Exp4 ("<="|"<"|"=="|"!="|">="|">") Exp4       -- binary
            | Exp4
  Exp4      = Exp4 ("+" | "-") Exp5                         -- binary
            | Exp5
  Exp5      = Exp5 ("*" | "/" | "%") Exp6                   -- binary
            | Exp6
  Exp6      = Exp7 "**" Exp6                                -- binary
            | Exp7
  Exp7      = num
            | true
            | false
            | id "(" ListOf<Exp, ","> ")"                   -- call
            | id                                            -- id
            | "(" Exp ")"                                   -- parens

  let       = "let" ~idchar
  function  = "function" ~idchar
  while     = "while" ~idchar
  true      = "true" ~idchar
  false     = "false" ~idchar
  print     = "print" ~idchar
  keyword   = let | function | while | true | false
  num       = digit+ ("." digit+)? (("E" | "e") ("+" | "-")? digit+)?
  id        = ~keyword letter idchar*
  idchar    = letter | digit | "_"
  space    += "//" (~"\n" any)*                             -- comment
}
```

Carlos

Carlos is another language for this class. It's a bit more sophisticated than Bella.

carlos.ohm

```
Carlos {
  Program     = Statement+

  Statement   = VarDecl
              | TypeDecl
              | FunDecl
              | Exp9 ("++" | "--") ";"                          --bump
              | Exp9 "=" Exp ";"                                --assign
              | Exp9_call ";"                                   --call
              | break ";"                                       --break
              | return Exp ";"                                  --return
              | return ";"                                      --shortreturn
              | IfStmt
              | LoopStmt

  VarDecl     = (let | const) id "=" Exp ";"
  TypeDecl    = struct id "{" Field* "}"
  Field       = id ":" Type
  FunDecl     = function id Params (":" Type)? Block
  Params      = "(" ListOf<Param, ","> ")"
  Param       = id ":" Type

  Type        = Type "?"                                        --optional
              | "[" Type "]"                                    --array
              | "(" ListOf<Type, ","> ")" "->" Type             --function
              | id                                              --id

  IfStmt      = if Exp Block else Block                         --long
              | if Exp Block else IfStmt                        --elsif
              | if Exp Block                                    --short
  LoopStmt    = while Exp Block                                 --while
              | repeat Exp Block                                --repeat
              | for id in Exp ("..." | "..<") Exp Block         --range
              | for id in Exp Block                             --collection
  Block       = "{" Statement* "}"
```

```
Exp         = Exp1 "?" Exp1 ":" Exp                          --conditional
            | Exp1
Exp1        = Exp1 "??" Exp2                                 --unwrapelse
            | Exp2
Exp2        = Exp3 ("||" Exp3)+                              --or
            | Exp3 ("&&" Exp3)+                              --and
            | Exp3
Exp3        = Exp4 ("|" Exp4)+                               --bitor
            | Exp4 ("^" Exp4)+                               --bitxor
            | Exp4 ("&" Exp4)+                               --bitand
            | Exp4
Exp4        = Exp5 ("<="|"<"|"=="|"!="|">="|">") Exp5        --compare
            | Exp5
Exp5        = Exp5 ("<<" | ">>") Exp6                        --shift
            | Exp6
Exp6        = Exp6 ("+" | "-") Exp7                          --add
            | Exp7
Exp7        = Exp7 ("*"| "/" | "%") Exp8                     --multiply
            | Exp8
Exp8        = Exp9 "**" Exp8                                 --power
            | Exp9
            | ("#" | "-" | "!" | some | random) Exp9         --unary
Exp9        = true ~mut
            | false ~mut
            | floatlit ~mut
            | intlit ~mut
            | no Type ~mut                                   --emptyopt
            | Exp9 ("(" | "?(") ListOf<Exp, ","> ")" ~mut    --call
            | Exp9 ("[" | "?[") Exp "]"                      --subscript
            | Exp9 ("." | "?.") id                           --member
            | stringlit ~mut
            | id                                             --id
            | Type_array "(" ")" ~mut                        --emptyarray
            | "[" NonemptyListOf<Exp, ","> "]" ~mut          --arrayexp
            | "(" Exp ")" ~mut                               --parens

intlit      = digit+
floatlit    = digit+ "." digit+ (("E" | "e") ("+" | "-")? digit+)?
stringlit   = "\"" char* "\""
```

```
  char        = ~control ~"\\" ~"\"" any
              | "\\" ("n" | "t" | "\"" | "\\")                --escape
              | "\\u{" hex hex? hex? hex? hex? hex? "}"       --codepoint
  control     = "\x00".."\x1f" | "\x80".."\x9f"
  hex         = hexDigit
  mut         = ~"==" "=" | "++" | "--"

  let         = "let" ~alnum
  const       = "const" ~alnum
  struct      = "struct" ~alnum
  function    = "function" ~alnum
  if          = "if" ~alnum
  else        = "else" ~alnum
  while       = "while" ~alnum
  repeat      = "repeat" ~alnum
  for         = "for" ~alnum
  in          = "in" ~alnum
  random      = "random" ~alnum
  break       = "break" ~alnum
  return      = "return" ~alnum
  some        = "some" ~alnum
  no          = "no" ~alnum
  true        = "true" ~alnum
  false       = "false" ~alnum
  keyword     = let | const | struct | function | if | else | while | repeat
              | for | in | break | return | some | no | random | true | false
  id          = ~keyword letter alnum*

  space      += "//" (~"\n" any)*                             --comment
}
```

Dax

Dax is a made up functional language.

dax.ohm

```
Dax {
  Program     = Exp
  Exp         = ("-" | "!") Primary                          --unary
              | Exp0 then Exp                                --then
              | Exp0
  Exp0        = Exp0 "|>" Exp1                               --pipe
              | Exp1
  Exp1        = Exp2 "?" Exp2 ":" Exp1                       --ternary
              | Exp2
  Exp2        = Exp3 ("&&" Exp3)+                            --and
              | Exp3 ("||" Exp3)+                            --or
              | Exp3
  Exp3        = Exp4 relop Exp4                              --compare
              | Exp4
  Exp4        = Exp4 ("+" | "-") Term                        --addsub
              | Term
  Term        = Term ("*" | "/" | "%") Factor                --muldiv
              | Factor
  Factor      = Primary "**" Factor                          --exponent
              | Primary
  Primary     = num
              | true
              | false
              | stringlit
              | let Dec (";" Dec)* in Exp kwend              --let
              | id Primary+                                  --id_call
              | id                                           --id
              | "(" Exp ")" Primary+                         --paren_call
              | "(" Exp ")"                                  --paren
              | "[" ListOf<Exp, ","> "]"                     --array
              | "{" id "=>" Exp "}" Primary+                 --lambda_call
              | "{" id "=>" Exp "}"                          --lambda
  Dec         = id "=" Exp
  relop       = "<=" | "<" | "==" | "!=" | ">=" | ">"
  then        = "then" ~idchar
  let         = "let" ~idchar
  in          = "in" ~idchar
  kwend       = "end" ~idchar
```

```
  true        = "true" ~idchar
  false       = "false" ~idchar
  keyword     = then | let | in | kwend | true | false
  id          = ~keyword letter idchar*
  idchar      = letter | digit | "_"
  num         = digit+ ("." digit+)? (("E" | "e") ("+" | "-")? digit+)?
  stringlit   = "\"" char* "\""
  char        = ~control ~"\\" ~"\"" any
              | "\\" ("n" | "t" | "\"" | "\\")              --escape
              | "\\u{" hex hex? hex? hex? hex? hex? "}"     --codepoint
  control     = "\x00".."\x1f" | "\x80".."\x9f"
  hex         = hexDigit
  space      += "//" (~"\n" any)* "\n"                     -- comment
}
```

A Java-like scripting language

Here is a small scripting language that is very Java-like in appearance, with a few simplifying features: (1) there are no classes, (2) the language needs little if any type inference, and (3) the standard library is both package-less and microscopic:

```
TinyJava {
  Program     =  Stmt+
  Stmt        =  Dec                          -- declaration
              |  Assignment ";"               -- assignment
              |  Call ";"                     -- call
              |  break ";"                    -- break
              |  return Exp? ";"              -- return
              |  print Args ";"               -- print
              |  if Exp Block
                 (else if Exp Block)*
                 (else Block)?                -- if
              |  while Exp Block              -- while
              |  for "("
                 (Type id "=" Exp)? ";"
                 Exp? ";"
                 Assignment? ")"
```

```
               Block                        -- for
Assignment  =  Increment                    -- increment
            |  Var "=" Exp                  -- plain
Increment   =  incop Var                    -- prefix
            |  Var incop                    -- postfix
Dec         =  TypeDec | VarDec | FunDec
TypeDec     =  struct id "{"
               (Type id ";")*
               "}"
Type        =  Type "[" "]"                 -- array
            |  primtype
            |  id
VarDec      =  Type id ("=" Exp)? ";"
FunDec      =  (Type | void) id "(" Params ")" Block
Params      =  (Type id ("," Type id)*)?
Block       =  "{" Stmt* "}"
Exp         =  NonemptyListOf<Exp1, "||">
Exp1        =  NonemptyListOf<Exp2, "&&">
Exp2        =  NonemptyListOf<Exp3, "|">
Exp3        =  NonemptyListOf<Exp4, "^">
Exp4        =  NonemptyListOf<Exp5, "&">
Exp5        =  Exp6 (relop Exp6)?
Exp6        =  NonemptyListOf<Exp7, shiftop>
Exp7        =  NonemptyListOf<Exp8, addop>
Exp8        =  NonemptyListOf<Exp9, mulop>
Exp9        =  prefixop? Exp10
Exp10       =  Literal
            |  Increment
            |  Var
            |  NewObject
            |  NewArray
            |  EmptyArray
            |  "(" Exp ")"                  -- parens
Literal     =  null
            |  true
            |  false
            |  floatlit
            |  intlit
            |  charlit
```

```
            |  stringlit
Var         =  Var "[" Exp "]"            -- subscript
            |  Var "." id                 -- select
            |  Call
            |  id
NewObject   =  new id "{" Args "}"
NewArray    =  new Type_array "{" Args "}"
EmptyArray  =  new Type "[" Exp "]"
Call        =  id "(" Args ")"
Args        =  ListOf<Exp, ",">

boolean     = "boolean" ~idchar
break       = "break" ~idchar
char        = "char" ~idchar
else        = "else" ~idchar
false       = "false" ~idchar
for         = "for" ~idchar
if          = "if" ~idchar
int         = "int" ~idchar
length      = "length" ~idchar
new         = "new" ~idchar
null        = "null" ~idchar
print       = "print" ~idchar
real        = "real" ~idchar
return      = "return" ~idchar
string      = "string" ~idchar
return      = "return" ~idchar
struct      = "struct" ~idchar
true        = "true" ~idchar
void        = "void" ~idchar
while       = "while" ~idchar

keyword     =  boolean | if | break | else | int | for | new
            |  return | char | struct | null | while | real
            | true | string | void | false | length | print
id          =  ~keyword letter idchar*
idchar      =  "_" | alnum
intlit      =  digit+
floatlit    =  digit+ "." digit+ (("E"|"e") ("+"|"-")? digit+)?
```

```
  char        =  escape
              |  ~"\\" ~"\"" ~"'" ~"\n" any
  h           =  hexDigit
  escape      =  "\\\\" | "\\\"" | "\\'" | "\\n" | "\\t" | hexseq
  hexseq      =  "\\" h h? h? h? h? h? h? h? ";"
  charlit     =  "'" (char | "\"") "'"
  stringlit   =  "\"" (char | "\'")* "\""
  addop       =  "+" | "-"
  relop       =  "<=" | "<" | "==" | "!=" | ">=" | ">"
  shiftop     =  "<<" | ">>"
  mulop       =  "*" | "/" | "%"
  prefixop    =  ~"--" "-" | "!" | "~" | char | int | string | length
  incop       =  "++" | "--"
  primtype    =  boolean | char | int | real | string

  space      :=  "\x20" | "\x09" | "\x0A" | "\x0D" | comment
  comment     =  "//" (~"\n" any)*
}
```

> ***Exercise**: Explain every line of this grammar. Note those things you find unconventional.*

JavaScript

Yes, there is a full ECMAScript grammar in Ohm out there.

Brainfuck

Brainfuck has a pretty trivial grammar. There are eight one-character instructions, and every other character is ignored.

```
Brainfuck {
  Program     = instruction+
  instruction = "[" instruction* "]"                -- loop
              | "," | "." | "+" | "-" | "<" | ">"
```

```
  space       = ~instruction any
}
```

> ***Exercise**: Later, after learning about semantics objects in Ohm, write a semantics for this grammar to create a compiler that translates Brainfuck into C. See the [Wikipedia article] for hints.*

Example Applications

Ohm can do so much more—so much more—than just generate ASTs. You can do just about anything in the semantics (parsing functions). Here are a few case studies:

Infix to Postfix Conversion

Here is a language of infix arithmetic expressions over numbers and identifiers. From lowest to highest the precedences are: (1) the left-associative additive operators, (2) left-associative multiplicative operators, (3) the prefix unary negation operator, (4) right-associative the exponentiation operator.

infix-to-postfix.mjs

```
import * as ohm from 'ohm-js'

const grammar = ohm.grammar(`Infix {
  Program = Exp
  Exp     = Exp ("+" | "-") Term     --binary
          | Term
  Term    = Term ("*" | "/") Factor  --binary
          | Factor
  Factor  = "-" Factor               --negate
          | Primary "**" Factor      --binary
          | Primary
  Primary = "(" Exp ")"              --parens
          | digit+                   --number
          | letter alnum*            --identifier
}`)
```

```
const semantics = grammar.createSemantics().addOperation('post', {
  Program(body) {
    return body.post()
  },
  Exp_binary(left, op, right) {
    return `${left.post()} ${right.post()} ${op.sourceString}`
  },
  Term_binary(left, op, right) {
    return `${left.post()} ${right.post()} ${op.sourceString}`
  },
  Factor_binary(left, op, right) {
    return `${left.post()} ${right.post()} ${op.sourceString}`
  },
  Factor_negate(_op, operand) {
    return `${operand.post()} ~`
  },
  Primary_parens(_open, expression, _close) {
    return expression.post()
  },
  Primary_number(_chars) {
    return this.sourceString
  },
  Primary_identifier(_first, _rest) {
    return this.sourceString
  },
})

export function toPostfix(infix) {
  let match = grammar.match(infix)
  if (!match.succeeded()) {
    throw new Error(match.message)
  }
  return semantics(match).post()
}
```

Unit tests:

infix-to-postfix-test.mjs

```
import { describe, it } from "node:test"
import assert from "assert/strict"
import { toPostfix } from "../infix-to-postfix.mjs"

describe("When converting infix to postfix", () => {
  it("works", () => {
    assert.equal(toPostfix("1 + 2"), "1 2 +")
    assert.equal(toPostfix("1 + 2 - 3"), "1 2 + 3 -")
    assert.equal(toPostfix("1 ** 2 ** 3"), "1 2 3 ** **")
    assert.equal(toPostfix("-1 ** 2 ** 3"), "1 2 3 ** ** ~")
  })
})
```

Run the tests with a command such as `node --test test/infix-to-postfix-test.mjs`

> ***Exercise**: Write a command line application that does the infix to postfix conversion, using the module above.*

Evaluating and Differentiating Polynomials 😍

This fun script illustrates multiple semantics for the same grammar.

polynomial.mjs

```
import * as ohm from 'ohm-js'

const grammar = ohm.grammar(`Polynomial {
  Poly
    = Poly "+" term              -- add
    | Poly "-" term              -- subtract
    | "-" Poly                   -- negate
    | term
  term
    = coefficient "x^" exponent  -- coeff_var_exp
    | coefficient "x"            -- coeff_var
    | coefficient                -- coeff
    | "x^" exponent              -- var_exp
```

```
    | "x"                          -- var
  coefficient
    = digit+ ("." digit+)?
  exponent
    = "-"? digit+
}`)

/* eslint-disable no-unused-vars */
const semantics = grammar
  .createSemantics()
  .addOperation('deriv', {
    Poly_add(p, op, t) {
      return `${p.deriv()}+${t.deriv()}`
    },
    Poly_subtract(p, op, t) {
      return `${p.deriv()}-${t.deriv()}`
    },
    Poly_negate(_, p) {
      return `-${p.deriv()}`
    },
    term_coeff_var_exp(c, _, e) {
      return `${e.value * c.value}x^${e.value - 1}`
    },
    term_coeff_var(c, _) {
      return `${c.value}`
    },
    term_coeff(c) {
      return '0'
    },
    term_var_exp(_, e) {
      return `${e.value}x^${e.value - 1}`
    },
    term_var(_) {
      return '1'
    },
  })
  .addOperation('eval', {
    Poly_add(p, op, t) {
      return x => p.eval()(x) + t.eval()(x)
```

```
    },
    Poly_subtract(p, op, t) {
      return x => p.eval()(x) - t.eval()(x)
    },
    Poly_negate(_, p) {
      return x => -p.eval()(x)
    },
    term_coeff_var_exp(c, _, e) {
      return x => c.value * x ** e.value
    },
    term_coeff_var(c, _) {
      return x => c.value * x
    },
    term_coeff(c) {
      return x => c.value
    },
    term_var_exp(_, e) {
      return x => x ** e.value
    },
    term_var(_) {
      return x => x
    },
  })
  .addAttribute('value', {
    coefficient(whole, dot, fraction) {
      return +this.sourceString
    },
    exponent(sign, magnitude) {
      return +this.sourceString
    },
  })

export function derivative(poly) {
  const match = grammar.match(poly)
  if (!match.succeeded()) {
    throw new Error(match.message)
  }
  return semantics(match).deriv()
    .replace(/\-\-/g, '+')
```

```
    .replace(/\+\-/g, '-')
    .replace(/^\+/, '')
    .replace(/\+0(?!\d)/, '')
    .replace(/\^1(?!\d)$/, '')
}

export function evaluate(poly, x) {
  const match = grammar.match(poly)
  if (!match.succeeded()) {
    throw new Error(match.message)
  }
  return semantics(match).eval()(x)
}
```

With unit tests even!

poly-test.mjs

```
import { describe, it } from "node:test"
import assert from "assert/strict"
import { derivative, evaluate } from "../polynomial.mjs"

describe("The differentiator", () => {
  it("detects malformed polynomials", () => {
    assert.throws(() => derivative("2y"))
    assert.throws(() => derivative("blah"))
    assert.throws(() => derivative("2x*6"))
  })
  it("correctly differentiates single-term polynomials", () => {
    assert.equal(derivative("4"), "0")
    assert.equal(derivative("2238"), "0")
    assert.equal(derivative("x"), "1")
    assert.equal(derivative("4x"), "4")
    assert.equal(derivative("x^5"), "5x^4")
    assert.equal(derivative("2x^-4"), "-8x^-5")
  })
  it("correctly differentiates multi-term polynomials", () => {
    assert.equal(derivative("-4"), "-0")
    assert.equal(derivative("-2238"), "-0")
    assert.equal(derivative("-x"), "-1")
```

```
    assert.equal(derivative("-x^5"), "-5x^4")
    assert.equal(derivative("-x^-5"), "5x^-6")
    assert.equal(derivative("2x^-4    + 7x^2"), "-8x^-5+14x")
    assert.equal(derivative("   2x^-4- 7x^20"), "-8x^-5-140x^19")
    assert.equal(derivative("2x^-4       +7x^-2"), "-8x^-5-14x^-3")
  })
})

describe("The evaluator", () => {
  it("detects malformed polynomials", () => {
    assert.throws(() => evaluate("2y"))
    assert.throws(() => evaluate("blah"))
    assert.throws(() => evaluate("2x*6"))
  })
  it("correctly evaluates single-term polynomials", () => {
    assert.equal(evaluate("4", 5), 4)
    assert.equal(evaluate("2238", 19), 2238)
    assert.equal(evaluate("x", 3), 3)
    assert.equal(evaluate("4x", 10), 40)
    assert.equal(evaluate("x^5", 2), 32)
    assert.equal(evaluate("32x^-4", 2), 2)
  })
  it("correctly evaluates complex polynomials", () => {
    assert.equal(evaluate("-4", 8), -4)
    assert.equal(evaluate("-2238", 11), -2238)
    assert.equal(evaluate("-x", 5), -5)
    assert.equal(evaluate("-x^5", 3), -243)
    assert.equal(evaluate("-x^-3", 2), -0.125)
    assert.equal(evaluate("2x^8    + 7x^2", 2), 540)
    assert.equal(evaluate("   2x^3- 7x^1", 4), 100)
    assert.equal(evaluate("2x^3       +7x^1", 4), 156)
  })
})
```

CLASSWORK

We'll build a command line application allowing differentiation and evaluation of polynomials. Should be fun!

More Examples

The Ohm repository on GitHub has a nice list of Examples.

Recall Practice

Here are some questions useful for your spaced repetition learning. Many of the answers are not found on this page. Some will have popped up in lecture. Others will require you to do your own research.

What can you do with Ohm? HIDE ANSWER

(1) Define the syntax and semantics of a language, and (2) easily write parsers, interpreters, compilers, and more.

Why is the Ohm Editor useful? HIDE ANSWER

It allows for an interactive programming language design by providing immediate feedback as you author the grammar.

What is the difference between `x` and `"x"` in Ohm? HIDE ANSWER

`x` is a rule, `"x"` is a literal.

What is the difference between lexical and syntax rules in Ohm? HIDE ANSWER

Lexical rules are for tokens, syntax rules are for phrases that are allowed to implicitly have spaces between components.

What does the parsing expression `e1 | e2` mean? HIDE ANSWER

Try to match `e1`, and if that fails, try to match `e2`.

What does the parsing expression `e*` mean? HIDE ANSWER

Match `e` zero or more times.

What does the parsing expression `e+` mean? HIDE ANSWER

Match `e` one or more times.

What does the parsing expression `e?` mean? HIDE ANSWER

Match `e` zero or one times.

What does the parsing expression `&e` mean? HIDE ANSWER

Match `e` without consuming it (positive lookahead).

What does the parsing expression `~e` mean? HIDE ANSWER

Succeed if `e` doesn't match, without consuming anything (negative lookahead).

What is the built-in rule that matches the end of the input in Ohm? HIDE ANSWER

```
end
```

How do you represent "any character except a newline" in Ohm? HIDE ANSWER

```
~"\n" any
```

Why does Ohm require each branch of an alternation to have the same arity? HIDE ANSWER

So that parsing actions can take a fixed number of parameters.

What cool shorthand does Ohm allow so that you don't have to explicitly created separate related rules to make sure arity restrictions are followed? HIDE ANSWER

You can use `--label` at the end of alternatives.

How does the `match` method on an Ohm grammar object work? HIDE ANSWER

It returns a `MatchResult` object that wraps a parse tree if the match succeeded.

What are the main methods and properties of an Ohm MatchResult? HIDE ANSWER

`succeeded()`, `failed()`, `message`, `shortMessage`.

How do you obtain a semantics object for a grammar *g* in Ohm? HIDE ANSWER

```
g.createSemantics()
```

What is the difference between operations and attributes in an Ohm semantics object? HIDE ANSWER

Operations are functions that are called every time, attributes are memoized and called with property syntax.

What are the three kinds of parse nodes in Ohm? HIDE ANSWER

Rule application nodes, iteration nodes, terminal nodes.

If you call an action method on a terminal node, but there is no associated method in the action dictionary, what happens? HIDE ANSWER

The special method `_terminal` is called, which by default returns `this.sourceString`.

If you call an action method on an iteration node, but there is no associated method in the action dictionary, what happens? HIDE ANSWER

You will get an error, unless you have properly defined an `_iter` method in the dictionary.

Is it a best practice to fall back to the `_iter` method? HIDE ANSWER

No, it is better to write `myNode.children.map(c => c.rep())` directly each time you have to process an iteration node.

Summary

We've covered:

- ✅ A good deal about Ohm
- ✅ Where to find Ohm's documentation
- ✅ Example Ohm Grammars
- ✅ Examples of semantic processing in Ohm